A Higher-Order Separation Logic for Verifying Liveness-Preserving Refinements of Distributed Systems

JONAS KASTBERG HINRICHSEN, IT University of Copenhagen, Denmark
LÉO STEFANESCO, University of Cambridge, England
LARS BIRKEDAL, Aarhus University, Denmark
AMIN TIMANY, Aarhus University, Denmark

Distributed systems are notoriously difficult to design, implement, and reason about. This is especially true for liveness properties which state that a desired state of the system is guaranteed to eventually be reached. A lot of complexity is found in distributed system implementations as these often employ features such as higher-order constructions and concurrency. Such features are difficult to formally reason about even without the added complexity introduced by the asynchronous communication present in distributed systems, or considering liveness properties. In this paper we present Fairneris, the first higher-order concurrent program logic for formal and foundational verification of safety alongside *liveness-preserving refinements* for distributed system implementations. This work builds on the recent success of the Iris-based Trillium/Fairis program logic framework for liveness-preserving refinement reasoning about higher-order concurrent programs, and that of the Aneris program logic for reasoning about (only) safety properties of distributed systems. To demonstrate the capabilities of Fairneris we verify both safety and liveness properties of an implementation of a higher-order distributed RPC-like system on top of unreliable networking primitives, through a refinement to a live high-level model. All results presented in the paper are mechanized on top of the Rocq proof assistant.

Additional Key Words and Phrases: Distributed systems, liveness, separation logic, refinement, higher-order logic, concurrency, formal verification, mechanization

ACM Reference Format:
Jonas Kastberg Hinrichsen, Léo Stefanesco, Lars Birkedal, and Amin Timany. 2026. A Higher-Order Separation Logic for Verifying Liveness-Preserving Refinements of Distributed Systems. 1, 1 (March 2026), 27 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Distributed systems can benefit significantly from formal verification, as they often serve critical functions, but are notoriously difficult to get right and to test exhaustively [11]. Traditionally, verification of distributed systems focused on reasoning about correctness of communication protocols, *e.g.*, using model checking tools like SPIN [12] and TLA+ [17]. However, implementations often have bugs in details that are not specified in high-level specifications [6]. Verifying distributed systems' implementations is particularly challenging, as modern distributed systems employ advanced features such as node-local concurrency and higher-order functions. Some recent work has taken up this challenge but they have only focused on *safety* [16] properties, *e.g.*, Aneris [15], Disel [27], Grove [28], Igloo [30], and Verdi [33]. A safety property is a property that asserts that nothing bad ever happens, *e.g.*, "the server never sends back an invalid response". The only exception is

Authors' Contact Information: Jonas Kastberg Hinrichsen, IT University of Copenhagen, , Denmark, hinrichsen@cs.au.dk; Léo Stefanesco, University of Cambridge, , England, leo.stefanesco@cl.cam.ac.uk; Lars Birkedal, Aarhus University, , Denmark, birkedal@cs.au.dk; Amin Timany, Aarhus University, , Denmark, timany@cs.au.dk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
© 2026 Copyright held by the owner/author(s).
ACM XXXX-XXXX/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

IronFleet [11] which also offers rather limited support reasoning about *liveness* [16] properties, *i.e.*, properties that state something good eventually happens, *e.g.*, "the server eventually responds to all requests". IronFleet supports a limited class of liveness properties for a limited class of programs by reducing liveness to termination of sequential programs. In IronFleet each node of the distributed system must be implemented as a simple single-threaded event loop—liveness is proved by showing that all event handlers terminate.

The key reason for the success of the state-of-the-art techniques for verification of implementations of concurrent and distributed systems is *modularity*. That is, in all these systems each module (function, method, class, thread, *etc.*), is proven correct against its own specification in isolation from other modules, under the assumption that the other modules are correct with respect to their own specification. Modular reasoning about liveness properties of concurrent programs is known to be notoriously challenging [7, 32]. This also carries over to distributed programs, even when each node is single-threaded. These challenges are only exacerbated by higher-order programming features, *e.g.*, first-class (higher-order) functions. In fact, among the state-of-the-art program logics for reasoning about liveness of concurrent programs, LiLi [20], TaDa-live [7], Lilo [19], and Fairis [32], only Fairis and Lilo support higher-order programming features, and only Fairis supports impredicativity. In this paper, we present **Fairneris**, the first program logic for verification of *both safety and liveness* properties of higher-order and concurrent implementations of distributed systems, under a flexible network fairness assumption. We also present the verification of safety and liveness of a case-study, higher-order remote procedure call (HORPC), which is a simple, minimal example demonstrating (the versatility of) Fairneris.

1.1 The Fairneris Program Logic

Fairneris builds on two program logics, Fairis [32], a program logic for verification of *both safety and liveness* properties of higher-order concurrent programs, and Aneris [15], a program logic for proving *only safety* properties of higher-order concurrent distributed systems, which crucially relies on impredicativity of its underlying logic Iris. Fairneris, just like both Fairis and Aneris, is formalized on top of the Iris framework [14], the state-of-the-art higher-order concurrent separation logic framework. Thus, Fairneris is the first program logic for *foundational* reasoning about *both safety and liveness* properties of arbitrary *higher-order* and *concurrent* programs that have access to network sockets. Here, by foundational, we mean that everything is mechanized in a proof assistant; the Rocq Prover in this case.

Following the methodology of Fairis/Trillium [32], Fairneris's approach to program verification is based on the notion of refinements. The idea, in a nutshell, is that to prove correctness of a distributed program Σ, we first pick a labeled transition system (LTS) $\mathcal{U}$, called the (user) model, and show that the program *refines* the model, written $\Sigma \preccurlyeq \mathcal{U}$, and separately show that the property of interest, *e.g.*, fair termination, holds for $\mathcal{U}$. The crucial point here is that the refinement relation $\preccurlyeq$ is what we call a *liveness-preserving* refinement, meaning that any liveness (and safety) property that the model $\mathcal{U}$ enjoys (possibly under some fairness conditions) are also satisfied by the distributed system Σ (under the same fairness conditions). Our main contribution in this work is the program logic of Fairneris, which allow us to establish the liveness-preserving refinement relation between the program and the user model. We prove the soundness of our approach in two theorems: the adequacy theorem of the program logic, *i.e.*, that a proof in the program logic implies $\Sigma \preccurlyeq \mathcal{U}$, and the soundness of the refinement relation, *i.e.*, that when $\Sigma \preccurlyeq \mathcal{U}$ holds and a liveness property holds for $\mathcal{U}$, then it also holds for Σ; see Theorem 2.4.

Integrating the reasoning principles for networking features present in Aneris into the refinement-based methodology Fairis involves solving a number of fundamental challenges. The **first challenge** is related to the unreliable nature of the network. In Fairneris's operational semantics network

messages can be non-deterministically dropped, duplicated, and reordered. Hence, liveness can only be guaranteed if the network is fair, *i.e.* it does not drop all messages. This in turn requires incorporating network fairness in our reasoning, both at the level of program semantics and at the level of the model. We will discuss these challenges and how we solve them in §2. The **second challenge** is related to maintaining the relation between the concrete state of the network in the operational semantics with the abstract network behavior in the model. Intuitively, in order to maintain the refinement, throughout the proof going over the program, whenever the program performs a network operation, it must be reflected in the model. Thus, in order to avoid reasoning directly about the states and transitions of the model throughout the proof, the abstract behavior of the network at the model level must be reflected in the program logic as logical formulae. For this purpose, we introduce a novel *model update* construction; see §3. The **third challenge** relates to reconciling the difference in the abstraction level of the network's behavior in the operational semantics and the network's behavior in the model; the former has far too many details that are irrelevant at the high abstraction level of the model, such as the network state. To address this challenge, as part of our soundness theorems, we construct a *tower of refinements* which we compose together to obtain the final refinement, and which step-by-step removes the irrelevant details of the network's behavior while maintaining enough information so we can prove preservation of liveness properties. Importantly, when reasoning about the abstract model, we can use invariants which have been proved with the program logic, so that safety reasoning is not repeated twice.

1.2 Case Study: Higher-Order RPC-like Distributed System

To demonstrate Fairneris's expressivity, we prove safety and liveness of a higher-order RPC-like distributed system, HORPC. The HORPC system consists of two higher-order functions that run on two different network nodes: the function running on one node takes as arguments a producer function p and a consumer function c; the function running on the other node takes as argument a transformer function t. The producer-consumer node repeatedly calls p to produce a new value v, which it sends over to the transformer node which computes and sends back $t(v)$. The producer-consumer node then takes $t(v)$ and calls c with it. This process repeats indefinitely, effectively computing $c(t(p()))$ repeatedly. The producer function p is stateful, and may thus produce different values at every call. The liveness property we prove for HORPC is that it will always eventually process a new value produced by p. The verification of HORPC exercises the core challenges of safety and liveness reasoning about distributed synchronization. It uses sequence ids, retransmissions, and serialization of values. Moreover, HORPC is live only if the functions p, t, and c are both safe and terminating. We illustrate how our logic handle these challenges. To the best of our knowledge, this is the first modular liveness proof of a higher-order distributed system implementation.

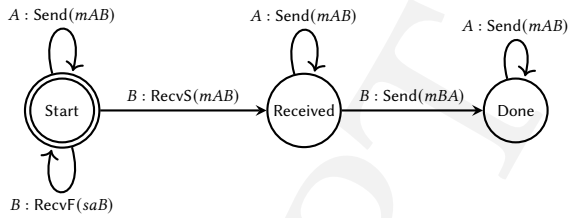
Contributions. In summary, in this paper we make the following contributions:

- We define new network fairness conditions at both the operational semantics level and the model level and discuss their relationship (§2).
- We present the Fairneris program logic to prove liveness-preserving refinement for distributed systems (§3). The logic includes a novel model update construction which lifts properties of the model's network actions into the program logic (§3).
- We present the first liveness proof of a higher-order implementation of a distributed system with modular composition of higher-order functions (§4).
- We build a tower of refinement relations which relate the detailed operational network semantics to the abstract network actions present in the high-level user model (§5.2).

All the result of the paper have been mechanized in the Rocq proof assistant, and can be found in our accompanying artifact [3].

```
nodeA saA saB =
  let sock = new_socket saA in
  while true do
    send sock saB "ping"

nodeB saA saB =
  let sock = new_socket saB in
  while recv sock = None do skip;
  send sock saA "done!"
```



$$mAB \triangleq (saA, \texttt{"ping"}, saB)$$
$$mBA \triangleq (saB, \texttt{"done!"}, saA)$$

Fig. 1. Retransmit example consisting of the program Σ_{retr} (left), and the model $\mathcal{U}_{retr}$ (right)

2 The Fairneris Approach to Liveness-Preserving Refinement

In this section we use a simple introductory example, to motivate and present the liveness-preserving refinement approach of Fairneris, alongside the fairness assumptions that we make, and demonstrate how these enable proving liveness properties of distributed systems. We first present the example (in §2.1). We then present how we define fairness and liveness as trace properties for both program traces (in §2.2) and model traces (in §2.3), and finally discuss how we use these notions to prove liveness of fair program traces, provided the liveness-preserving refinement $\preccurlyeq$ (in §2.4).

2.1 Retransmit Example

The example consists of two nodes, A and B, where A continuously *retransmits* the message "ping" to B. The code for the two nodes, A and B, is given in Figure 1. We will write Σ_{retr} for this distributed system. The liveness property we wish to prove for Σ_{retr} is that node B eventually sends "done!" to node A. In Σ_{retr}, node A repeatedly sends the message "ping" to node B, while node B waits for this by repeatedly calling **recv**, before sending back a "done!" and terminating. Our operational semantics models an unconnected *unreliable* network. Nodes create socket handlers with **new_socket**, which binds it to the given socket address, *e.g.* saA or saB. Using the socket handle **send** puts messages in a network message soup, after which they can be non-deterministically dropped, duplicated, or delivered (out of order). Delivered messages are put in the receive buffer of the designated recipient socket, after which they can be retrieved by performing a receive operation, **recv**, on a corresponding socket handler. As a result, it is possible for a message to never be delivered, preventing liveness. However, *under network fairness assumptions*, node B must eventually send "done!" to node A.[1] As we explained in §1, in Fairneris we prove liveness by establishing a refinement between the distributed program and a high-level user-picked model. For our simple example here the model $\mathcal{U}_{retr}$, seen in Figure 1, is given. The model $\mathcal{U}_{retr}$ has three states: Start, the initial state, corresponds to the situation where node A is sending "ping", Received, where node B has received it at least once, but has not yet replied, and, finally, Done where node B has sent back "done!". Note $\mathcal{U}_{retr}$ is significantly simpler than Σ_{retr}. In particular, due to the non-deterministic thread scheduling, non-deterministic behavior of the network dropping and duplicating messages, the loop in node A, *etc.*, the state space of Σ_{retr} is not even finite. By contrast, $\mathcal{U}_{retr}$ has only three states, and completely abstracts over the network state and the receive buffers.

[1]Note that it is impossible to have two nodes simultaneously reach agreement. This phenomenon, known as the two-generals problem [2, appendix] occurs even when the network is completely reliable, as soon as there is any delay in communication. Hence, here we only show that one node, node B reaches the end of communication.

2.2 Fairness and Liveness in the Program Σ_{retr}

All throughout this paper, we will use *linear temporal logic* (LTL) formulas to state properties about potentially infinite traces of labelled-transition systems corresponding to the program and user-picked models. In a few words, we use the LTL operators "eventually", written $\mathbb{F}P$, which means that P must eventually hold at some point in the trace, and "always", written $\mathbb{G}P$, which means that P should hold at all points in the trace. We describe network events in terms of the actions: Send, Deliver, RecvS, and RecvF, ranged over by α. The $\mathsf{Send}(saA, \mathrm{m}, saB)$ action stands for saA sending the message m to saB, $\mathsf{Deliver}(saA, \mathrm{m}, saB)$ stands for delivery of a message m from saA to saB's receive buffer, $\mathsf{RecvS}(saA, \mathrm{m}, saB)$ stands for saB receiving the message m (*i.e.*, retrieving it from saB's buffer) from saA, and $\mathsf{RecvF}(saB)$ stands for saB attempting to perform a receive which fails due to the receive buffer being empty. We often use an action α as syntactic sugar for an LTL proposition, denoting that the network action of the labelled transition corresponds to α. The liveness property of Σ_{retr} explained above can be formally stated as follows:

$$\text{For all executions } ex \text{ of } \Sigma_{retr},\ \ ex \vDash \mathbb{F}\ \mathsf{Send}(\mathsf{mBA}) \tag{Live-Wrong}$$

Of course, this property, as stated in (Live-Wrong) does not hold unconditionally because in our network semantics, messages can be non-deterministically dropped. Thus, we need to include some network fairness assumptions. A natural network fairness condition is the following:

$$\forall saA, msg, saB.\ \ \mathbb{G}\big(\mathbb{G}\mathbb{F}\ \mathsf{Send}(saA, msg, saB) \implies \mathbb{F}\ \mathsf{Deliver}(saA, msg, saB)\big) \tag{NetFair-Weak}$$

which states that any message sent infinitely often must eventually be delivered. This condition is indeed sufficient for Σ_{retr}, but it is too weak for many applications as it requires the exact same message to be retransmitted to guarantee delivery. For example, it would be too weak if we consider a variant of Σ_{retr} where the retransmission is tagged with a counter representing how many times the message has been retransmitted. This variant would never retransmit the same message from saA to saB, rendering (NetFair-Weak) insufficient. A similar situation appears in our case study, presented in §4. As a result, we work with a stronger notion of network fairness, (NetFair) below:

$$\begin{aligned}&\forall saA, \Phi, saB.\\ &\quad \mathbb{G}\,(\mathbb{G}\mathbb{F}\ (\exists m.\ \Phi(m) \wedge \mathsf{Send}(saA, m, saB)) \implies \mathbb{F}\ (\exists m.\ \Phi(m) \wedge \mathsf{Deliver}(saA, m, saB)))\end{aligned} \tag{NetFair}$$

Intuitively, (NetFair) states that for any set of messages $(\{m \mid \Phi(m)\})$, such that if we infinitely often send some message in this set, then at least one such message must be delivered. We emphasize that (NetFair) implies (NetFair-Weak); simply take Φ to be a singleton $\{msg\}$. In our hypothetical variant of the program in Figure 1, which tags messages with how many times they have been retransmitted, we can take Φ to be the set of all messages consisting of "ping" together with an arbitrary retransmission tag. In that case, (NetFair) would imply that at least one message with body "ping" is eventually delivered with some retransmission tag.

In addition to fairness of the network, the property (Live-Wrong) relies on each node being represented in the trace infinitely often. If A, which keeps performing **send**, is not represented "frequently enough", we cannot satisfy the assumption of (NetFair), and therefore not guarantee eventual delivery. In the Fairneris's operational semantics, like Aneris's, we treat all threads of all nodes as one large thread pool. A benefit of this is that we can implicitly capture fair representation of nodes as a consequence of fair scheduling of threads. We borrow this fair scheduling assumption directly from Fairis, stated as the following LTL property

$$\forall \zeta.\ \ \mathbb{G}\big(\mathsf{thread_enabled}(\zeta) \implies \mathbb{F}\ \mathsf{thread_steps}(\zeta)\big) \tag{SchedFair}$$

where the predicate $\mathsf{thread_enabled}(\zeta)$ means that the locale ζ (a pair of node id and a thread id in that node) is not terminated, *i.e.* if a thread can run, then it eventually runs. We are now able to

state the correct result about Σ_{retr}, as a general liveness property over predicates on actions:

$$\text{For all executions } ex \text{ of a distributed system } \Sigma, \quad ex \vDash \big(\mathsf{SchedFair} \wedge \mathsf{NetFair}\big) \implies \mathbb{F}\,\alpha \qquad \text{(Live)}$$

Where $\Sigma := \Sigma_{retr}$ and $\alpha := \mathsf{Send}(\mathsf{mBA})$. This is the statement that we eventually prove about the system Σ_{retr} using our program logic by first lifting it to the level of $\mathcal{U}_{retr}$ in §2.3.

Remark 2.1 (Locality of Liveness). The property (Live) above is local in the sense that it states that for any trace *ex*, if *ex* is fair, then eventually α. This means that the program logic neither needs to enforce fairness conditions, nor does it need to rely on them. The program logic simply establishes a refinement relation between programs and models, including both fair and unfair executions. Crucially, the refinement relation still enable derivation of liveness properties. Thus, showing fair termination/liveness of model traces implies fair termination/liveness of program traces.

Remark 2.2 (Fairness and Delivery in the Presence of Epochs). Network fairness is subtle: even though both (NetFair-Weak) and (NetFair) are predicated on messages being sent infinitely often, doing so is not strictly necessary, such as in the presence of epochs, *i.e.*, where a message is only retransmitted *until* it is acknowledged. Indeed, it suffices to only guarantee retransmission until an acknowledgement is received. Consider the two cases: either we receive an acknowledgement, in which case we know the message is delivered and thus need not retransmit anymore. Or, we never receive any acknowledgement. In that case, we keep retransmitting the message forever, which as per fairness, means that the other party receives the message infinitely often. But then, since the other party sends an acknowledgement back every time it receives our message, we have that the acknowledgement is sent infinitely often. Thus, by fairness we must receive an acknowledgement which is contradictory. We leverage this observation in the proof of liveness of our case study in §4.

2.3 Fairness and Liveness in the User Model $\mathcal{U}_{retr}$

As we explained in §1, our methodology involves showing a liveness-preserving refinement between the program (here Σ_{retr}) and the model (here $\mathcal{U}_{retr}$), $\Sigma_{retr} \preccurlyeq \mathcal{U}_{retr}$. Based on this liveness-preserving refinement we can transport a liveness property from the model to the program. For our example in Figure 1, the model counterpart of (Live) can be stated via the following general liveness property for models over predicates on network actions, where $\mathcal{U} := \mathcal{U}_{retr}$ and $\alpha := \mathsf{Send}(\mathsf{mBA})$:

$$\text{For all traces } utr \text{ of a user model } \mathcal{U}, \quad utr \vDash \big(\mathsf{ModSchedFair} \wedge \mathsf{ModNetFair}\big) \implies \mathbb{F}\,\alpha \quad \text{(ModLive)}$$

ModSchedFair and ModNetFair, which we will define and explain below, are respectively the model counterparts to SchedFair and NetFair.

The labels in the model $\mathcal{U}$ are of the form $\rho : \alpha$ where ρ is a role and α is some model network action. Roles ρ, an invention of Fairis [32], are an abstraction of threads. The idea (covered in detail in §3) is that each thread is assigned a number of roles which both allow and mandate the thread to take transitions in those roles. On the model side, fairness of the scheduler is expressed via roles:

$$\mathbb{G}\big(\mathsf{role_enabled}(\rho) \implies \mathbb{F}\ \mathsf{role_steps}(\rho)\big) \qquad \text{(ModSchedFair)}$$

Here, $\mathsf{role_enabled}(\rho)$ means that the current state has a transition with a label whose role is ρ, and $\mathsf{role_steps}(\rho)$ means that a transition is taken with a label whose role is ρ. Note the resemblance of this fairness condition to (SchedFair). The refinement relation mandates that when a thread is assigned a role, that thread can only postpone taking a transition in that role for finitely many steps. This is the reason why (SchedFair) implies (ModSchedFair). In our example, the main threads of nodes A and B are respectively assigned roles A and B in $\mathcal{U}_{retr}$. The second part of the labels, the

network action α, is used to express network fairness at the model level:

$$\forall saA, saB, \Phi.\ \mathbb{G}\begin{pmatrix} \mathbb{GF}\ (\exists m.\ \Phi(m) \wedge \mathsf{Send}(saA, m, saB)) \implies \\ \mathbb{GF}\ \mathsf{Recv_from}(saB) \implies \\ \mathbb{F}\ (\exists m.\ \Phi(m) \wedge \mathsf{RecvS}(saA, m, saB)) \end{pmatrix} \tag{ModNetFair}$$

Note how this fairness condition resembles (NetFair), except for the fact that delivery is replaced with receiving, and the extra condition $\mathbb{GF}\ \mathsf{Recv_from}(saB)$. In this condition, $\mathsf{Recv_from}(saB) \triangleq \mathsf{RecvF}(saB) \vee \exists saA, m.\ \mathsf{RecvS}(saA, m, saB)$ is a shorthand for attempting to receive. Thus, the fairness condition (ModNetFair), in addition to requiring messages to be repeatedly sent, also requires that an attempt is also frequently made to receive.

Before we proceed to explain how (NetFair) implies (ModNetFair), we will first explain how (ModLive) holds for $\mathcal{U}_{retr}$ in Figure 1 with $\alpha := \mathsf{Send}(mBA)$. As per scheduling fairness, it is clear that it is sufficient to reach the Received state, as we cannot repeatedly take the transition $A : \mathsf{Send}(mAB)$ in that state, and hence must eventually take the transition by the role B which sends the "done!" message. In turn, proving that Received is reached amounts to proving that the transition labeled with $\mathsf{RecvS}(mAB)$ is eventually taken. This follows directly from (ModNetFair). Assume that this transition is never taken. In that case, we stay in the state Start forever. However, staying in Start forever, by fairness of scheduling, satisfies the two premises of (ModNetFair). Hence, the transition $\mathsf{RecvS}(mAB)$ must eventually be taken, which contradicts our assumption.

As we explained earlier, the Deliver network action does not appear in the user-picked model $\mathcal{U}$. This is a result of a discrepancy between the network semantics of the program and the model. As explained in §2.1 the program network semantics is based on receive buffers. This is inherited from the choice of basing the Fairneris network semantics on those of Aneris. Given receive buffers, a natural fairness assumption is then to capture eventual delivery in terms of the Deliver action. Despite this discrepancy, there is no cost w.r.t. the user experience, as our liveness-refinement $\preccurlyeq$ transports the fairness assumption all the same. Intuitively, to show that (NetFair) implies (ModNetFair), we essentially have to show delivered messages are received, *i.e.*, we have to show that receive operations are performed, which follows from the condition $\mathbb{GF}\ \mathsf{Recv_from}(saB)$ in (ModNetFair), which we further detail in §5.3.

Remark 2.3 (Alternative network semantics). An alternative to receive buffers and delivery actions is a network semantics that removes the receive buffers, and instead have the **recv** instruction receive messages directly from the network message soup, with the option of failing non-deterministically. This permits a program-level fairness assumption akin to the user model one; that repeated attempted receives $\mathsf{Recv_from}(saB)$ will eventually result in success $\mathsf{RecvS}(saA, m, saB)$. However, this approach would require us to redefine the Aneris network semantics, and consequently would let us to inherit less of the Aneris proof infrastructure. Secondly, while the alternative network semantics is closer the user model, it would not reduce the once-and-for-all refinement effort significantly. More generally, it shows that it is not necessary to change the specifications to facilitate verification, instead we can use a model which closely reflects the actual systems.

2.4 Liveness-Preserving Refinement Theorem

In addition to prior mentions, the refinement relation can include a user-picked model invariant $\Psi : \mathcal{U} \to \mathsf{Prop}$, written $\preccurlyeq_\Psi$. This invariant, which is proved using the program logic, enable deriving liveness properties from model traces that exclude states which do not satisfy Ψ, written $utr \setminus \Psi$. This yields flexibility in the choice of user model, which may then sidestep some details in their construction; an idea that we use in §4. We omit the model invariant whenever it is trivial

$\Psi\, m \triangleq$ True. With the definitions presented throughout this section we now give the formal theorem for deriving (action-based) liveness properties via the liveness-preserving refinement $\Sigma \preccurlyeq_\Psi \mathcal{U}$.

Theorem 2.4 (Liveness-Preserving Refinement). *Whenever $\Sigma \preccurlyeq_\Psi \mathcal{U}$ holds, then for any program trace ex, starting in Σ, there exists a user trace utr, starting in the initial model state, such that if* (ModLive) *holds for the user trace, then* (Live) *holds for the program trace.*

Note that the fairness assumption on the program trace is preserved upwards (from program to model trace), while the liveness property is preserved downwards (from model to program trace).

3 The Fairneris Program Logic

In this section we present the Fairneris program logic. We first present the adequacy theorem (§3.1), which states that, from a proof of weakest preconditions of all system nodes, we can conclude the liveness-preserving refinement $\preccurlyeq_\Psi$. We then present the rules of Fairneris (§3.2) which are used to prove weakest preconditions. Finally, we remark on how the Fairneris logic is used to verify the refinement of the retransmit example from Figure 1 (§3.3).

To prove our liveness-preserving refinement we must (1) prove safety of the distributed system, (2) ensure only finite stuttering program steps between model steps for each role, and (3) guarantee correspondence between program and model. To solve (1), we adopt the approach to reasoning about distributed systems from Aneris [15]. To solve (2), we adopt the idea of a *fuel-instrumentation* from Fairis [32], which, as explained in §2.3, ensures productivity by assigning roles to threads, and obligating each thread to take at most a user-picked finite number of stuttering steps F_{cap} in between every productive step, for each role assigned to them. To solve (3), we extend the Fairis approach to model correspondence with support for *non-deterministic* actions—such as **recv** that emits either $\mathsf{RecvS}(m)$ or $\mathsf{RecvF}(sa)$—by exposing the actions taken by the network operations in the logic, allowing the model step to depend on the action emitted by the program. This is a strict improvement over the Fairis rules for model steps, that require choosing the model step before resolving the program step. The key technical contribution of Fairneris is our solution to (3)—the "model update" construction—which allows us to decompose model conformance from program verification (§3.2), and thereby avoid rule duplication (See Remark 3.3).

3.1 Fairneris Adequacy

The Fairneris program logic is a separation logic built on top of the notion of *weakest preconditions*. A weakest precondition $\mathsf{wp}^{\zeta}\ e\ \{\Phi\}$ intuitively means that the expression e is safe to execute, and if it terminates with value w, then $\Phi\ w$ holds. In Fairneris, weakest preconditions are instrumented with a locale $\zeta \in Ip \times \mathbb{N}$, denoting the distributed system node id (modelled as an ip address) that the expression belongs to, along with its thread id on that node.

The Fairneris adequacy theorem formalises that the refinement $\preccurlyeq_\Psi$ holds, if we can prove (1) weakest preconditions for all main threads of all nodes, and (2) that the user-picked model invariant Ψ always holds, denoted: $\mathit{AlwaysHolds}(\Psi)$. To carry out the proof, we are given the initial separation logic resources describing the initial state for: (i) the model state (*à la* Trillium); (ii) the fuel-instrumentation (*à la* Fairis); (iii) the network state (*à la* Aneris); and (iv) allocate initial invariants. The initial invariants are imperative, as they help us to prove the model invariant between the program and model states. The formal statement of the adequacy theorem is as follows:

Theorem 3.1 (Fairneris-Adequacy). *Let Σ be a node-pool ($Ip \rightharpoonup Expr$) of the initial system nodes. Let $\mathsf{m} \in \mathcal{U}$ be the initial model state of $\mathcal{U}$, and Ψ be a predicate on model states ($\mathcal{U} \to Prop$). Let F_{init} be an initial fuel map ($Ip \to (\mathbf{role} \to \mathbb{N})$) with all $ip \in \mathrm{dom}(\Sigma)$ assigned to respective role maps fs, mapping the roles of m to a user-picked fuel limit F_{cap}. All roles of m are uniquely covered in the*

initial fuel map. Let A be a map (Address → SProt) from all known socket addresses of the network to a socket protocol that must be satisfied for inbound messages (SProt ≜ Message → Prop). If

$$\begin{pmatrix} \circ_{\gamma_{\mathcal{M}}}(\mathsf{m}) * (\mathop{\ast}_{ip \mapsto fs \in \mathsf{F}_{init}} (ip, 0) \Longrightarrow fs) * \\ \mathop{\ast}_{sa \mapsto \Phi_{sa} \in A} \mathsf{Unbound}(sa) * sa \Rrightarrow \Phi_{sa} * sa \rightsquigarrow (\emptyset, \emptyset) \end{pmatrix} \mathrel{-\!\!*}$$
$$\Rrightarrow \mathit{AlwaysHolds}(\Psi) * \mathop{\ast}_{ip \mapsto e \in \Sigma} wp^{(ip,0)}\ e \left\{ (ip, 0) \Longrightarrow \emptyset \right\})$$

then $\Sigma \preccurlyeq_{\Psi} \mathcal{U}$ holds in the meta-logic.

Here $\circ_{\gamma_{\mathcal{M}}}(\mathsf{m})$ reflects the state of the user model (i), $(\mathop{\ast}_{ip \mapsto fs \in \mathsf{F}_{init}} (ip, 0) \Longrightarrow fs)$ the state of the fuel-instrumentation of Fairis (ii), and $(\mathop{\ast}_{sa \mapsto \Phi_{sa} \in A} \mathsf{Unbound}(sa) * sa \Rrightarrow \Phi_{sa} * sa \rightsquigarrow (\emptyset, \emptyset))$ the initial network resources of Aneris (iii). The modality $\Rrightarrow P$ denotes the permission to update ghost state before proving P, including establishing invariants. Here, we use it to allocate the initial invariants (iv), based on the initial resources, which are then enforced to hold at all times by the logic's proof rules when proving individual WPs, and thereby available when proving $\mathit{AlwaysHolds}(\Psi)$. We elide explicit details about the creation, opening, and closing of invariants in this paper, as it is standard w.r.t. Iris-based separation logics. Instead, we remark that invariants can safely be allocated at any time, and accessed during every step of program execution, provided they are left unchanged [14]. We cover each of these constructs in more detail in the following sections.

Remark 3.2 (Using natural numbers for fuel). Soundness requires that fuel be ordinals smaller than the step indexing ordinal of the underlying logic Iris. We inherit natural numbers from Fairis. Given transfinite indexing [29], it would be possible to have arbitrary ordinals as fuel, although it is unclear if the transfinite variant is compatible with the rest of Trillium, and thus our work.

3.2 Verification in Fairneris

The proof obligation of the Fairneris logic is to verify the model invariant $\mathit{AlwaysHolds}(\Psi)$ and the weakest preconditions. We first cover how the former is achieved via the invariants of Fairneris inherited from Iris, and then how the latter is achieved via the Fairneris weakest precondition rules.

Iris Invariants and Model Invariant. Fairneris, like Trillium and Iris before it, makes use of separation logic invariants to determine global properties $\boxed{P}$ that hold in between any step of execution. In separation logic, invariants are primarily used for safely sharing otherwise exclusive resources between threads. In Trillium, and thereby Fairneris we additionally use the global properties enforced by the invariants to verify the user-picked model invariant Ψ. In particular, the user has to prove $\mathit{AlwaysHolds}(\Psi)$, which permits access to the contents of all invariants, and discerning the state of the model via the available separation logic resources, alongside a global view of the user model state $\bullet_{\gamma_{\mathcal{M}}}(m)$. This is made precise by the following rules:

$$\frac{\forall \mathsf{m}.\ \bullet_{\gamma_{\mathcal{M}}}(\mathsf{m}) \mathrel{-\!\!*} \Rrightarrow \bullet_{\gamma_{\mathcal{M}}}(\mathsf{m}) * \Psi\ \mathsf{m}}{\mathit{AlwaysHolds}(\Psi)}* \qquad \begin{array}{l}\textsc{Mod-agree}\\ \bullet_{\gamma_{\mathcal{M}}}(m) * \circ_{\gamma_{\mathcal{M}}}(m') \vdash m = m'\end{array}$$

For example, with an initial invariant such as $\boxed{\exists m,\ \circ_{\gamma_{\mathcal{M}}}(m) * \Phi\ m}$, we can (1) open the invariant, (2) deduce the exact state of the model using Mod-agree, and (3) use $\Phi\ m$ to prove $\Psi\ m$.

Fairneris Weakest Precondition Rules. The weakest precondition of Fairneris captures that a program is safe to execute, and that the program refines the user model. Following the principles of Trillium, the role of the program logic is to prove an initial lockstep refinement; that the program is in one-to-one correspondence with a stutter-instrumented user model. The lockstep refinement is then automatically further refined into the $\preccurlyeq_{\Psi}$ refinement of the user model, yielded by the adequacy theorem, as will be explained in §5.2. The proof of the weakest precondition mimics the

Weakest Precondition and Model Update rules, and excerpt of SSWP Rules

WP-STEP

$$\frac{\mathsf{wp}^{\zeta}\ e \left\langle e'; \alpha^{?}.\ \mathsf{mu}^{\zeta;\alpha^{?}}\ \mathsf{wp}^{\zeta}\ e'\ \{\Phi\}\right\rangle}{\mathsf{wp}^{\zeta}\ e\ \{\Phi\}}*$$

WP-ROLE-DEALLOC

$$\frac{\circ_{\gamma_{\mathcal{M}}}(\mathsf{m}) \qquad \mathsf{m} \overset{\rho\ ;}{\not\longrightarrow} _ \qquad \zeta \Rrightarrow \{\rho := _\} \uplus fs \qquad (\circ_{\gamma_{\mathcal{M}}}(\mathsf{m}) * \zeta \Rrightarrow fs) \mathrel{-\!\!*} \mathsf{wp}^{\zeta}\ e\ \{\Phi\}}{\mathsf{wp}^{\zeta}\ e\ \{\Phi\}}*$$

MDL-STEP-FUEL

$$\frac{\zeta \Rrightarrow fs^{+1} \qquad fs \neq \emptyset \qquad (\zeta \Rrightarrow fs \mathrel{-\!\!*} P)}{\mathsf{mu}^{\zeta}\ P}*$$

MDL-STEP-MODEL

$$\frac{\circ_{\gamma_{\mathcal{M}}}(\mathsf{m}) \qquad \mathsf{m} \xrightarrow{\rho\ :\ \alpha} \mathsf{m}' \qquad \zeta \Rrightarrow \{\rho := _\} \uplus (fs^{+1}) \qquad ((\circ_{\gamma_{\mathcal{M}}}(\mathsf{m}') * \zeta \Rrightarrow \{\rho := \mathsf{F}_{cap}\} \uplus fs) \mathrel{-\!\!*} P)}{\mathsf{mu}^{\zeta;\alpha}\ P}*$$

WP-ROLE-FORK

$$\frac{\zeta \Rrightarrow (fs_1 \uplus fs_2)^{+1} \qquad fs_1 \uplus fs_2 \neq \emptyset \qquad \forall \zeta'.\ \zeta'.ip = \zeta.ip \mathrel{-\!\!*} \zeta' \Rrightarrow fs_2 \mathrel{-\!\!*} \mathsf{wp}^{\zeta'}\ e\ \{\zeta' \Rrightarrow \emptyset\}}{\mathsf{wp}^{\zeta}\ \mathbf{fork}\ (e)\ \{\zeta \Rrightarrow fs_1\}}$$

SSWP-CREATE-SOCKET

$$\frac{\zeta.ip = sa.ip \qquad \mathsf{Unbound}(sa)}{\mathsf{wp}^{\zeta}\ \mathbf{new_socket}\ sa \left\langle sh.\ sh \hookrightarrow sa\right\rangle}*$$

SSWP-SEND

$$\frac{\zeta.ip = msg.\mathsf{src.ip} \qquad sh \hookrightarrow msg.\mathsf{src} \qquad msg.\mathsf{dst} \Mapsto \Phi \qquad msg.\mathsf{src} \rightsquigarrow (R, T) \qquad (msg \notin T \Rightarrow \Phi\ msg)}{\mathsf{wp}^{\zeta}\ \mathbf{send}\ sh\ msg.\mathsf{str}\ msg.\mathsf{dst} \left\langle w; \alpha. \begin{array}{l} w = |msg.\mathsf{str}| * \alpha = \mathsf{Send}(msg) * \\ msg.\mathsf{src} \rightsquigarrow (R, T \cup \{msg\}) * sh \hookrightarrow msg.\mathsf{src} \end{array}\right\rangle}*$$

SSWP-RECV

$$\frac{\zeta.ip = sa.\mathsf{ip} \qquad sh \hookrightarrow sa \qquad sa \rightsquigarrow (R, T) \qquad sa \Mapsto \Phi}{\mathsf{wp}^{\zeta}\ \mathbf{recv}\ sh \left\langle w; \alpha. \begin{array}{c} (w = \mathbf{None} * \alpha = \mathsf{RecvF}(sa) * sh \hookrightarrow sa * sa \rightsquigarrow (R, T)) \vee \\ (\exists msg.\ w = \mathbf{Some}\ (msg.\mathsf{str}, msg.\mathsf{src}) * \alpha = \mathsf{RecvS}(msg) * msg.\mathsf{dst} = sa * \\ sh \hookrightarrow sa * sa \rightsquigarrow (R \cup \{msg\}, T) * (msg \notin R \Rightarrow \Phi\ msg)) \end{array}\right\rangle}*$$

Fig. 2. The Fairneris Program Logic. Note, $\frac{P \quad Q}{R}*$ desugars to $\vdash ((P * Q) \mathrel{-\!\!*} R)$.

lockstep structure. At every step of the program, we must (1) prove that the program step is safe to execute, and (2) that we can either stutter or update the model correspondingly. To achieve this reasoning principle we decompose the proof correspondingly, by employing two constructions:

- The single-step weakest precondition (SSWP): $\mathsf{wp}^{\zeta}\ e\ \langle \Phi \rangle$
- The model update construction (MU): $\mathsf{mu}^{\zeta;\alpha^{?}}\ P$

The SSWP is inspired by its use in the Fairis logic, and VMSL [21] before it. It captures that the expression e is safe to execute for *one* step, after which the predicate Φ holds for the resulting expression e'. In Fairneris it additionally emits any action $\alpha^{?}$ exhibited by the step. Here, $\alpha^{?}$ signifies an optional action; we write $\bot$ and α to signify the absence and presence of an action, respectively. The MU is novel. It captures that the model is currently *out-of-sync* with the program by one step. It is parametric in the emitted action $\alpha^{?}$, denoting that we must update the model based on the *non-deterministic* results of the SSWP. The decomposition is imperative as it lets us define and prove rules for each construction independently of each other, ultimately avoiding rule duplication, as we remark below. With the decomposition in place, we obtain the rules of Fairneris shown in

Figure 2. We use the following syntactic sugar:

$$\mathsf{wp}^{\zeta}\, e\, \{w.\, Q\} \triangleq \mathsf{wp}^{\zeta}\, e\, \{\lambda w.\, Q\} \qquad \mathsf{wp}^{\zeta}\, e\, \langle w; \alpha.\, Q\rangle \triangleq \mathsf{wp}^{\zeta}\, e\, \langle \lambda(w, \alpha).\, Q\rangle$$

$$\mathsf{wp}^{\zeta}\, e\, \{Q\} \triangleq \mathsf{wp}^{\zeta}\, e\, \{\lambda w.\, w = () * Q\} \qquad \mathsf{wp}^{\zeta}\, e\, \langle w.Q\rangle \triangleq \mathsf{wp}^{\zeta}\, e\, \left\langle \lambda(w, \alpha^{?}).\, \alpha^{?} = \bot * Q\right\rangle$$

$$\mathsf{mu}^{\zeta}\, P \triangleq \mathsf{mu}^{\zeta;\bot}\, P \qquad \mathsf{wp}^{\zeta}\, e\, \langle Q\rangle \triangleq \mathsf{wp}^{\zeta}\, e\, \left\langle \lambda(w, \alpha^{?}).\, w = () * \alpha^{?} = \bot * Q\right\rangle$$

The rules of Fairneris combine and extend the prior reasoning principles of Fairis [32] and Aneris [15], with the action exhibiting behaviour. In particular, we borrow the ideas for handling fuel from Fairis, while borrowing the ideas for reasoning about networks from Aneris.

The centerpiece of the decomposition is the WP-STEP rule. The rule states that to prove a WP, we can (1) prove a single step of the program via the SSWP rules, (2) reestablish consistency with the model via the MU, and (3) prove the rest of the program via the remaining WP. The SSWP construction allows access to all invariants, while the MU conversely requires reestablishing them, although we elide certain details for brevity sake. This permits accessing invariants for the combined SSWP/MU step in WP-STEP. This is crucial as resources maintained by invariants may apply to both the safety proof (governed by SSWP) and the model update (governed by MU).

The presented SSWP rules are inspired by Aneris [10, 15]; instrumented with action labels. The SSWP-CREATE-SOCKET rule covers allocation of new socket handlers *sh*, bound to a given unbound (ensured by $\mathsf{Unbound}(sa)$) socket address *sa*. As socket allocation is not represented in the model, no action is emitted. The SSWP-SEND and SSWP-RECV rules tie the emitted action to the corresponding operation, *e.g.* $\alpha = \mathsf{RecvF}(sa)$ when **recv** fails. Additionally, they follow the Aneris approach to sharing resources over an unreliable network, via *socket protocols* $sa \Mapsto \Phi$—initially established in initial configuration of the adequacy theorem—that describe the resources $\Phi\, m$ required for sending a message m to socket address *sa*. The resources are provided by the sender when sending the *first* message m to *sa*. Similarly, the resources are obtained upon the *first* reception of the message m by *sa*. The uniqueness of messages are tracked by the *message histories* $sa \rightsquigarrow (R, T)$, which track the set of received R and transmitted T messages for the socket address *sa*.

The MU rules are inspired by the Fairis model rules for fuel and model steps. The MDL-STEP-FUEL allow resolving non-model (unlabeled) "stuttering" steps, by burning non-zero fuel (denoted by $fs^{+n} \triangleq map\,(+n)\, fs$) for the relevant locale ζ. The MDL-STEP-MODEL allow resolving model steps, where the model transition has to reflect the action of the MU at a role ρ of the relevant locale ζ. That is, if the MU has a send label, then the model must take a send transition. If the MU does not have a label, the model must take an unlabeled transition. This means any network transition of the program *must* be reflected by the *user model*. The model step rule requires burning non-zero fuel for all other roles at locale ζ, while refueling the step-taking role ρ to the fuel limit F_{cap}. The WP-ROLE-DEALLOC rule allows for deallocation of roles once they have no transitions (and thus have concluded their lifetime), which is used to reach the post-condition of terminating programs, such as node B of the retransmit example. Finally, the WP-ROLE-FORK rule resolves **fork**, used to start new threads on a node. The rule requires us to split the fuel/role map of the current thread, and let each thread continue with one map each. The rule facilitates reasoning about node-local concurrency, similar to how concurrency was handled in Fairis. The rule is special as it does not allow for decomposition via MU, as updating the model state has to occur before the fork is resolved.

Remark 3.3 (Benefits of Decomposition). Without the rule WP-STEP alongside the decomposed rules SSWP-RECV, MDL-STEP-FUEL, and MDL-STEP-MODEL, we would need non-rules à la wp-recv-step-fuel and wp-recv-step-model, with the combined details of the model update and program-level instruction. Additionally, the wp-recv-step-model rule would have to include all the model-level details in the post-condition, so it can depend on the non-deterministic action emitted by **recv**.

3.3 Verification of Retransmit Example

With the Fairneris logic in hand, we can now verify the liveness-preserving refinement of the retransmit example $\Sigma_{retr} \preccurlyeq \mathcal{U}_{retr}$. As per the adequacy theorem, to show Σ_{retr} refines $\mathcal{U}_{retr}$ we can prove the following two Hoare triples (defined as standard: $\{P\}^{\zeta}\ e\ \{\Phi\} \triangleq P \mathrel{-\!\!*} \mathsf{wp}^{\zeta}\ e\ \{\Phi\}$):

$$\begin{Bmatrix} I_{retr} * \zeta_A \Longrightarrow \{A := 42\} * \mathsf{Unbound}(sa_A) * \\ sa_B \Longrightarrow (_ = mAB) * sa_A \rightsquigarrow (\emptyset, \emptyset) \end{Bmatrix}^{\zeta_A} \text{nodeA saA saB} \{\zeta_A \Longrightarrow \emptyset\}$$

$$\begin{Bmatrix} I_{retr} * \zeta_B \Longrightarrow \{B := 42\} * \circ_{\gamma_B}(\mathsf{Start}) * \mathsf{Unbound}(sa_B) * \\ sa_B \Longrightarrow (_ = mAB) * sa_A \Longrightarrow (_ = mBA) * sa_B \rightsquigarrow (\emptyset, \emptyset) \end{Bmatrix}^{\zeta_B} \text{nodeB saA saB} \{\zeta_B \Longrightarrow \emptyset\}$$

where the invariant I_{retr} is as follows:

$$I_{retr} \triangleq \boxed{\exists m,\ \circ_{\gamma_{\mathcal{M}}}(m)\ *\ \bullet_{\gamma_B}(m)}$$

The proofs follow almost immediately from symbolic execution using the decomposition rules and the SSWP rules of Fairneris. The auxiliary steps (such as the reduction of the while-loops) are simply resolved via burning fuel. The model steps are resolved by accessing the invariant to obtain the model state resource, and then taking the corresponding transitions in the model, simultaneously letting us re-fuel to resolve future auxiliary steps.

The key proof detail here is that the invariant I_{retr} states that we are, at all times in some state m of $\mathcal{U}_{retr}$, indicated by the resource $\circ_{\gamma_{\mathcal{M}}}(m)$ in I_{retr}. The resource $\bullet_{\gamma_B}(m)$ in I_{retr} and $\circ_{\gamma_B}(\mathsf{Start})$, in the precondition above, allow the thread ζ_B to keep track of the current state of $\mathcal{U}_{retr}$, while it is shared with thread ζ_A via the invariant. The resources $\bullet_{\gamma_B}(m)$ and $\circ_{\gamma_B}(m')$, readily available from Iris, can only be allocated and updated in tandem, and always agree on the state they track. This is codified in the logic as the following two rules:

OWN-ALLOC
$\vdash \Longrightarrow \bullet_{\gamma}(m) * \circ_{\gamma}(m)$

OWN-UPDATE
$\bullet_{\gamma}(m) * \circ_{\gamma}(m) \vdash \Longrightarrow \bullet_{\gamma}(m') * \circ_{\gamma}(m')$

OWN-AGREE
$\bullet_{\gamma}(m) * \circ_{\gamma}(m') \vdash m = m'$

Node A simply performs send operations in a loop; hence, each send follows the previous one after finitely many steps. Thus, we only need to show that each send operation performs an A-action. For this we need to know that we have the permission/obligation to do so. This is reflected in the fact that the A node is assigned role A, through $\zeta_A \Longrightarrow \{A := 42\}$. By the invariant I_{retr} we know that we are in some state m, indicated by $\circ_{\gamma_{\mathcal{M}}}(m)$. Since all states have an $A : \mathsf{Send}(mAB)$ transition, we always can/should perform the send operation. Furthermore, all these transitions are loops and hence never change the state tracked by $\circ_{\gamma_{\mathcal{M}}}(m)$, always maintaining the invariant.

For node B, initially, we know that we are in state Start, as we have $\circ_{\gamma_B}(\mathsf{Start})$, and by OWN-AGREE, the m in the invariant I_{retr} must be Start. In this state, taking the loop $B : \mathsf{RecvF}(saB)$ is completely analogous to taking $A : \mathsf{Send}(mAB)$. However, when the Recv operation succeeds, we go from Start to Received in $\mathcal{U}_{retr}$. That is, after taking that step we will have $\circ_{\gamma_{\mathcal{M}}}(\mathsf{Received})$. At this point, we will update resources using the rule OWN-UPDATE to obtain $\bullet_{\gamma_B}(\mathsf{Received})$ and $\circ_{\gamma_B}(\mathsf{Received})$ which maintains the invariant I_{retr}. Finally, in state Received we have only one permission/obligation, *i.e.*, to take the transition $B : \mathsf{Send}(mBA)$. We perform this action, and update our state to $\bullet_{\gamma_B}(\mathsf{Done})$ and $\circ_{\gamma_B}(\mathsf{Done})$. In this final state there are no B-obligations left and hence the thread *must* terminate (after a finite number of steps), because there is no action left in its assigned role.

With the proven weakest preconditions, we can use the Fairneris adequacy theorem to obtain the refinement $\preccurlyeq$ between the distributed system and the retransmit model shown in Figure 1. The theorem provides the resources needed for satisfying the preconditions of the WP proofs. The retransmit example does not rely on any model invariant, thus $AlwaysHolds(\Psi)$ is trivial.

4 Case Study: Higher-Order RPC-like Distributed System

To demonstrate the expressive power of Fairneris, we verify eventual progress for an implementation of a higher-order distributed RPC-like system, HORPC. We will first present HORPC's implementation and the formal liveness property that we prove in §4.1. We then give an LTS model for HORPC which captures the network protocol and the liveness property in §4.2. Finally, in §4.3, we conclude the proof of the liveness property at the implementation level through a proof of refinement between the implementation and the model via Fairneris.

4.1 HORPC and its Implementation

The HORPC is a higher-order program. The implementation can be found in Figure 3. The HORPC consists of two nodes: a sender node (node A in the figure) and a receiver node (node B in the figure). The program running on the sender node takes two functions: a producer function, p, and a consumer function, c. The program running on the receiver node takes a transformer function, t. The idea is that the coordination between the two nodes results in values being produced and consumed corresponding to evaluating the expression c(t(p())) repeatedly.

Remark 4.1 (Language is stateful). Note that since the language is stateful, repeated calls to functions, *e.g.*, p, may produce different results, incidentally impacting the calls to both t and c.

The sender node sends the sequence $(0, v_0), (1, v_1), (2, v_3), \ldots$ (where v_i is the result of the ith call to p ()), and receives the sequence $(0, w_0), (1, w_1), (2, w_3), \ldots$, where w_i is the result of (t v_i). Since the network is unreliable, the sender waits for the ith reply, *i.e.*, a message of the form (i, v), before sending $(i + 1, v_{i+1})$. On the sender side the situation is simple: as long as it has not received the ith reply from the receiver node for the current iteration, i, it retransmits the value (i, v_i) again. When it does receives the right id-value pair, *i.e.*, (i, w_i), it consumes w_i by calling c w_i, and proceeds by producing the next value v_{i+1} through the producer function, p, which it sends to the receiver node alongside the next number $i + 1$ in the next iteration of the loop (after the recursive call to go). The receiver node's behavior is almost the dual of the sender's: it repeatedly sends (j, w_j) for the highest id j it has yet received. When it receives the next id, $(j + 1, v_{j+1})$, it computes w_{j+1} by calling t v_{j+1}, which it sends to the sender node as a pair $(j + 1, w_{j+1})$. It then increments its internal sequence counter, and continues. The subtlety is that it only retransmits (j, w_j) when it receives a message that is not the next input $j + 1$ that it expects. Thus, in this example, if the sender node is not live, *i.e.*, it does not keep retransmitting, then the receiver node is not either. For brevity's sake, we omit details about serialization and deserialization which are required by our network semantics, that only permits sending strings over the network, not arbitrary values.

Similar to the retransmission example, the HORPC only makes progress when we consider fair scheduling and a fair network. Formally, the property that we want to prove is:

Lemma 4.2. *For all executions ex of Σ_{HORPC}, and natural numbers i,*

$$ex \vDash (\mathit{SchedFair} \wedge \mathit{NetFair}) \implies (\mathbb{F}\ \mathsf{Send}(\mathsf{mBA}\ \mathsf{i}\ _))$$

4.2 HORPC Model, Liveness Property, and Proof

To formally model the HORPC, we model the sender and receiver individually, and construct the user model as the cartesian product of the two, *i.e.*, the set of states is the cartesian product of sets of states with no synchronization on labels. The individual models can be found in Figure 3. We model the state of the sender as its current sequence id i, and whether it is in sending ($\top$) or receiving ($\bot$) state. Similarly, we model the state of the receiver as its current acknowledgement id j, and whether it is in sending ($\top$) or receiving ($\bot$) state. The initial state of the model is $((0, \top), (-1, \bot))$.

```
HORPC_A p c saA saB =
  let shA = new_socket saA in
  socket_bind shA saA;
  (rec go i v =
    send shA saB (i,v);
    match recv shA with
    | None   => go i v
    | Some m => let (j,w) = m.1 in
                if i = j
                then c w;
                     go (i+1) (p ())
                else go i v) 0 (p ())
```

```
HORPC_B t saA saB =
  let shB = new_socket saB in
  socket_bind shB saB;
  (rec go j w =
    match recv shB with
    | None   => go j w
    | Some m => let (i,v) = m.1 in
                if j+1 = i
                then let w := t v in
                     send shB saA (i,w);
                     go i w
                else send shB saA (j,w);
                go j w) -1 ()
```

$$mAB\ i\ v \triangleq (saA, (\mathrm{i}, v), saB) \qquad mBA\ j\ w \triangleq (saB, (\mathrm{j}, w), saA)$$

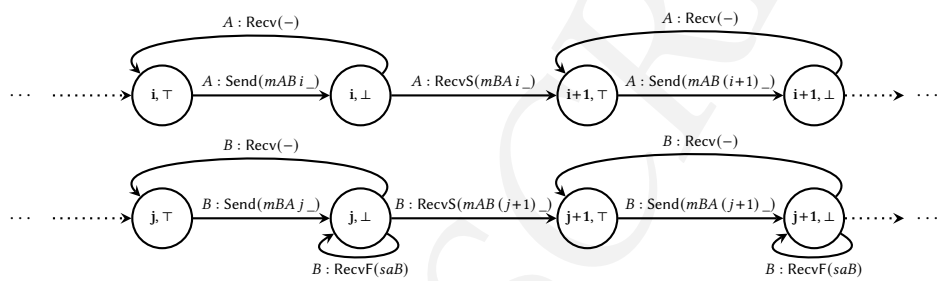


Fig. 3. The implementation (Σ_{HORPC}) and model ($\mathcal{U}_{HORPC}$) of the two nodes of the HORPC example.

The transitions of the model correspond to the transitions of the protocol. For example, when the sender is in state $(i, \bot)$ and receives a message with the expected acknowledgement $(i, _)$, it transitions to $(i + 1, \top)$. Conversely, when the receiver is in state $(j, \bot)$ and receives a message with the expected sequence id $(j + 1, _)$, it transitions to $(j + 1, \top)$. We use Recv$(-)$ to denote wildcard receive transitions matching any uncovered receive transition, including failure to receive.

It is important to note that the model is agnostic to the message values (indicated by wildcards _). This is beneficial, as they are irrelevant from the perspective of the liveness proof, and consequently, not restricting them allows us to refine strictly more programs. For example, one might want to extend the HORPC with an acknowledgement counter, as discussed in §2, and reuse the same model for the liveness proof. As a consequence of this flexibility, we do not know, or care, that retransmitted values are always the same. This requires us to use the stronger fairness condition (NetFair) in §2. Thus, we pick the set Φ in (NetFair) as the set $\Phi\ (i', v) \triangleq i = i'$, for the current sequence id i.

A caveat of the model in Figure 3 is that it does not rule out invalid states where the sender and receiver are out of sync, *i.e.* where the sequence id and acknowledgement id are more than one apart, as there is no connection between send and receive events. This is an issue, as traces that reach these invalid states do not satisfy the liveness property we want in general. However, these states are not reachable when the semantics of the network are taken into account. We could prove this safety property in a model with network semantics, but that would be a lot of manual work, and we already have a very good tool for proving safety properties of programs: the invariants of the program logic! As such, we can use the safety property $\mathbb{G}$AtMostOneOff below when reasoning about the model. That is, we prove the following property about the model $\mathcal{U}_{HORPC}$:

Lemma 4.3. *For all trace utr of $\mathcal{U}_{HORPC}$, and natural numbers i,*

$$utr \vDash (ModSchedFair \wedge ModNetFair) \implies \mathbb{G}\mathsf{AtMostOneOff} \implies (\mathbb{F}\ \mathsf{Send}(\mathsf{mBA}\ i\ _))$$

where AtMostOneOff *is a predicate on states (pairs of states of the sender and receiver):*

$$\mathsf{AtMostOneOff}((i, _), (j, _)) \triangleq i = j \vee i + 1 = j$$

4.3 HORPC Refinement in Fairneris

To transport the liveness property for the model to the implementation, we need to prove a refinement between the implementation and the model, with the user-picked model invariant (the Ψ in Theorem 3.1) being the predicate AtMostOneOff, which directly allows us to conclude the assumption $\mathbb{G}$AtMostOneOff in §4.2. At the program logic level we use the following invariant:

$$I_{HORPC} \triangleq \boxed{\exists i, stA, j, stB.\ \circ_{\gamma_M}((i, stA), (j, stB)) * \bullet_{\gamma_A}(stA) * \bullet_{\gamma_B}(stB) * \bullet_{\gamma_i}(i) * \bullet_{\gamma_j}(j) * (i = j \vee i + 1 = j)}$$

Similar to the retransmit example, the invariant owns the model state via $\circ_{\gamma_M}((i, stA), (j, stB))$, and exposes the individual parts of the state via the remaining ghost state. Note how the invariant above directly enforces the model invariant, AtMostOneOff. What remains is to prove the weakest preconditions of the sender and receiver nodes in the presence of the invariant above, and to apply the Fairneris adequacy theorem, Theorem 3.1.

We first have to consider the function arguments: p, t, and c. We ascribe specifications for these functions which capture that they are (1) terminating, (2) non-interfering w.r.t. the model, and (3) interconnected; they can be called one after the other (indicated by the fact that the postcondition of one is the precondition of the next in the call order):

$$\{\mathsf{True}\}^{\zeta}_{p_f}\ p\ ()\ \{v.\ \Phi_1\ v\} \quad \{\Phi_1\ v\}^{\zeta}_{t_f}\ t\ v\ \{w.\ \Phi_2\ w\} \quad \{\Phi_2\ w\}^{\zeta}_{c_f}\ c\ w\ \{\mathsf{True}\}$$

$$\text{where} \quad \{P\}^{\zeta}_{f}\ e\ \{\Phi\} \triangleq \forall fs.\ \{\zeta \Longrightarrow fs^{+f} * P\}^{\zeta}\ e\ \{v.\ \zeta \Longrightarrow fs * \Phi\ v\}$$

We specify (1) and (2) by requiring the presence of the fuel resource, with enough fuel to resolve the function-specific finite number of steps f: $\zeta \Longrightarrow fs^{+f}$. As the set of roles fs does not change between the precondition and postcondition, the specifications implicitly capture that the functions do not interfere with the LTS model of HORPC.

Remark 4.4 (Example instantiation of function parameters). An example instantiation of the functions, that satisfy our required specifications is the following:

```
p := let l := ref 0 in λ _. FAA l 2
t v = if v % 2 = 0 then v * 2 else abort
c w = if w % 2 = 0 then () else abort
```

Here, FAA is an atomic fetch-and-add instruction which returns the value of the location before performing the addition. The producer function p generates a stream of even integers 0, 2, 4, 6, ..., satisfying the postcondition $\Phi_1(x) \triangleq \mathtt{even_int}\ x$. The transformer function t tests that the input is even (which it should be since it should satisfy $\Phi_1(x)$), and doubles it in that case, otherwise it crashes by calling abort. Therefore, we expect t to satisfy the postcondition $\Phi_2(x) \triangleq \mathtt{even_int}\ x$. Finally, the consumer function c tests that the input is even, and otherwise crashes. Crucially, if the producer was not well-behaved, the transformer could crash, leading to liveness failing.

What remains to verify the HORPC implementation is to appropriately delegate the local views of the state between the sender and receiver nodes so that they can adequately make progress, while maintaining the invariant. Notably, the sender and receiver nodes need to deduce that any increment to their counter does not invalidate the invariant. Thus, at the logical level, they each

need to know what state the other is in whenever they make progress. This is achieved via the so-called *fractional ownership*, which we here apply to the ownership of the state. These fractional permissions satisfy the following rules:

OWN-FRAC-SPLIT
$$\circ_\gamma(m) \dashv\vdash \circ_\gamma^{1/2}(m) * \circ_\gamma^{1/2}(m)$$

OWN-FRAC-AGREE
$$\circ_\gamma^{1/2}(m) * \circ_\gamma^{1/2}(m') \vdash m = m'$$

That is, we can split a local view of the state in two parts, and unify values between them. With this we can enforce that the sender and receiver nodes "take turns" incrementing their respective part of the state, effectively enforcing the property AtMostOneOff. In particular, when verifying correctness of a node, we borrow half of the other node's local view of the state in order to make progress. When one node has successfully incremented its counter, it "yields the permission to progress" by sending its borrowed half local view, along with half of its own view, when sending a fresh message to the other party. We can achieve this passing of resources among the two nodes, as well as transporting the resources corresponding of the functions' preconditions and/or postconditions, via Aneris's socket protocols. The communication protocols for the receiver node, Φ_A, and the sender node Φ_B are as follows:

$$\Phi_A\ msg \triangleq \exists j, w.\ msg = \mathtt{mBA}\ j\ w * \circ_{\gamma_i}^{1/2}(j) * \circ_{\gamma_j}^{1/2}(j) * \Phi_2\ w$$
$$\Phi_B\ msg \triangleq \exists i, v.\ msg = \mathtt{mAB}\ i\ v * \circ_{\gamma_i}^{1/2}(i) * \circ_{\gamma_j}^{1/2}(i-1) * \Phi_1\ v$$

Note that the fractional ownership prohibits one from updating the ownership in the invariant.

With the invariant and socket protocols we only need to prove the following weakest preconditions (presented as Hoare triples):

$$\left\{\begin{array}{l} I_{HORPC} * sa_A \Rrightarrow \Phi_\mathsf{A} * sa_B \Rrightarrow \Phi_\mathsf{B} * sa_A \rightsquigarrow (\emptyset,\emptyset) * \mathsf{Unbound}(sa_A) * \\ \zeta_A \Rrightarrow \{A := \mathsf{F}_{cap}\} * \circ_{\gamma_i}(0) * \circ_{\gamma_j}^{1/2}(-1) * \circ_{\gamma_A}(\top) \end{array}\right\}_{\zeta_A} \begin{array}{l}\text{HORPC_A} \\ \quad \text{p c saA sAB}\end{array} \{\zeta_A \Rrightarrow \emptyset\}$$

$$\left\{\begin{array}{l} I_{HORPC} * sa_A \Rrightarrow \Phi_A * sa_B \Rrightarrow \Phi_B * sa_B \rightsquigarrow (\emptyset,\emptyset) * \mathsf{Unbound}(sa_B) * \\ \zeta_B \Rrightarrow \{B := \mathsf{F}_{cap}\} * \circ_{\gamma_j}^{1/2}(-1) * \circ_{\gamma_B}(\bot) \end{array}\right\}_{\zeta_B} \begin{array}{l}\text{HORPC_B} \\ \quad \text{t saA sAB}\end{array} \{\zeta_B \Rrightarrow \emptyset\}$$

The proof of the weakest preconditions is relatively straightforward, and mostly follows symbolic execution. The primary proof effort is to consolidate the fractional views to unify (using the rule OWN-FRAC-SPLIT) and update the model state at the appropriate times, to maintain the invariant. Note that we pick the fuel limit F_{cap} after choosing the functions p, t, and c. As a result, we can always pick enough fuel to handle their fuel obligation.

With the weakest preconditions in hand, we can finally apply the adequacy theorem. Similarly to the retransmit example, we can allocate all the necessary resources up front, and use it when allocating the invariant. From this, it is trivial to observe that the assumptions of the weakest preconditions follow directly from the initial resources provided by the adequacy theorem.

With the refinement and the $\mathbb{G}$(AtMostOneOff) safety property in hand, along with the proof of the liveness property at the model level, we can derive the liveness property for the implementation of the HORPC, completing the proof.

5 Inner Workings of Fairneris

This section explains *how* the Fairneris logic is constructed, and how its adequacy theorem is established.

The adequacy theorem of Fairneris, Theorem 3.1 establishes $\Sigma \preccurlyeq_\Psi \mathcal{U}$, for some distributed system Σ and model $\mathcal{U}$. More specifically, as will be made clear in this section, this means that for any program trace *ex* starting in Σ there *exists* a model trace *utr*, starting in the initial state of $\mathcal{U}$, where *ex* is refined by *utr* wrt. a liveness-preserving *trace* refinement. The crux of the adequacy

theorem is how *utr* is constructed, such that it upholds the trace refinement. The trace *utr* is not constructed directly, rather, the program logic, behind the scenes, constructs a trace *ftr* in the model Fuel($\mathcal{U} \ltimes \mathcal{N}et$), and then extracts *utr* out of *ftr*. Here, $\mathcal{N}et$ is a network model, $\ltimes$ is a synchronized product between two models, and Fuel is a fuel-construction on models, which will be explained in detail in this section. Below, we will first explain the operational semantics of our distributed system (§5.1). We then explain how this gave rise to the composite user-network model $\mathcal{U} \ltimes \mathcal{N}et$ (§5.2). We then present the *refinement tower* for constructing the user model trace $\mathcal{U}$ from Fuel($\mathcal{U} \ltimes \mathcal{N}et$) (§5.3). We finally present how the adequacy theorem of Fairneris was proven using a slightly modified version of Trillium (§5.4).

5.1 Concrete Operational Semantics of Distributed Systems

The Trillium framework [32], just like the Iris framework, does not fix the programming language — hence, both Fairis and Fairneris can be constructed on top of it. In practice, this means that the Trillium framework introduces its own notion what a programming language is. This allows instantiating Trillium by constructing an instance of this notion. In this work, we have made a simple adjustment to this notion of a programming language: we have introduced *actions*. This addition allows us to appropriately abstract the network's behavior as we will discuss below.

A programming language for Trillium is defined by fixing a set of terms, *Term*, a set of states, *State*, a set of locales, *Locale*, a set of user actions, **Usract**, and a set of system actions, **Sysact**, together with the following two relations for program steps and system steps:

$$\begin{aligned} \rightarrow_{\mathsf{prg}} &\subseteq (\mathit{Term} \times \mathit{State}) \times \mathbf{Usract}^{?} \times (\mathit{Term} \times \mathit{State} \times \mathit{Term}^{\star}) \\ \rightarrow_{\mathsf{sys}} &\subseteq \mathit{State} \times \mathbf{Sysact} \times \mathit{State} \end{aligned}$$

Terms are program expressions (AnerisLang expressions in case of AnerisLang). States describe the state of the entire system (for AnerisLang: a memory heap and a map of socket handles for each node, together with a so-called message soup of messages in transit). Locales are conceptual locations where programs run (a pair of an IP address together with a thread id for AnerisLang). A tuple $(e, \sigma, \alpha^{?}, e', \sigma', \vec{e}_f) \in \rightarrow_{\mathsf{prg}}$ describes a single step of the program reducing the term e to the term e' and updating the state from σ to σ' while forking threads in the finite sequence $\vec{e}_f$ of expressions — in AnerisLang $\vec{e}_f$ is only non-empty for the fork operation. The α label is optional (denoted by ?), to allow for internal transitions of the program, which do not involve the network. A tuple $(\sigma, \beta, \sigma') \in \rightarrow_{\mathsf{sys}}$ describes systems' autonomous updates; dropping, delivering, or duplicating in-transit messages in AnerisLang.

Based on these, Trillium defines so-called configurations as a pair of a thread pool (a mapping from locales to terms) and a state, $\mathit{Cfg} \triangleq \mathit{ThPool} \times \mathit{State}$, and defines a global step relation

$$\rightarrow_{glob} \subseteq \mathit{Cfg} \times ((\mathit{Locale} \times \mathbf{Usract}^{?}) \cup \mathbf{Sysact}) \times \mathit{Cfg}$$

The relation $\rightarrow_{glob}$ is defined by non-deterministically picking a locale and running it for one step, or by non-deterministically picking a system step. This describes all behaviors of the system including fair and unfair behaviors of the scheduler and the network.

For AnerisLang, the program steps and system steps are defined in a standard way of OCaml-like programming language with UDP-like networking primitive [15]. Here, we only describe the state in AnerisLang as it is relevant to the discussion in this paper. See Krogh-Jespersen et al. [15] for more details. In AnerisLang, the global state of a distributed system is of the form $((\mathcal{S}, \mathcal{M}), \mathcal{H})$, where $\mathcal{H}$ is a map associating each node $ip \in \mathit{Ip}$ with a program heap $h = \mathcal{H}(ip)$. The message soup $\mathcal{M}$ is a multiset of messages in transit. The socket state map $\mathcal{S}$ is a mapping from IP addresses to *local socket state* S, which itself is a map from *socket handlers* to pairs $(\mathit{skt}, \mathit{buf})$ of the receive buffer buf, and the *socket* itself skt, which contains the socket address $\mathit{skt.sa}$. Additionally, there

are some well-formedness conditions, for example: $\mathcal{S}[ip] = S$ and $S[sh] = (skt, buf)$ imply that the IP address of the socket address of *skt* is equal to *ip*.

5.2 Model Abstraction for Network Traces

Fairneris adequacy, Theorem 3.1, concludes the existence of a refinement between fair executions of the distributed system and traces of the user provided model $\mathcal{U}$. As we will see in §5.4, the adequacy theorem of the underlying Trillium logic constructs a refinement between the distributed system and a more complicated model, that contains both the user model $\mathcal{U}$ and an abstract description of the network. This section explains how this abstract network is combined with the user model $\mathcal{U}$, and how the refinement $\preccurlyeq_\Psi$ is defined.

Synchronized product. Our global model is the combination (obtained using the operation $\ltimes$) of the user model, chosen by the user, and the network model, which is fixed. For the sake of clarity, we first explain how the operation $\ltimes$ is defined in general, for an arbitrary instantiation of Trillium. That is, we define the composition of a user model with a *system model*, which abstracts the *system* from the previous section. The two notions of model differ slightly, as only user models keep track of *roles*, since the fairness of the network and scheduler are of different nature.

Definition 5.1 (System model). Consider a Trillium language instantiation with set of states *State* and set of system actions **Sysact**. A *system model* is a LTS $\mathcal{N}$ labeled with **Sysact**, together with a relation env_match and a map appSys:

$$\mathsf{env_match} \subseteq \mathit{State} \times \mathcal{N} \qquad \mathsf{appSys} : \mathcal{N} \to \mathbf{Sysact} \to \mathcal{N}$$

The relation env_match expresses that the concrete state of the system $\sigma \in \mathit{State}$, refines an abstract state $n \in \mathcal{N}$. The appSys specifies how the system state evolves when the system makes an autonomous action, *i.e.*, it reflects the relation $\to_{\mathsf{sys}}$. The above data also needs to satisfy some properties which we elide here.

The purpose of the appSys function is to preserve the refinement env_match between the distributed system and the abstract model when the network takes an autonomous step. This function is fixed because system steps are not under the control of the program, and so should be invisible to the user of the logic. A *user model* is essentially an LTS with specific labels:

Definition 5.2 (User model). Given a Trillium language instantiation with set of user actions **Usract**, a *user model* $\mathcal{U}$ is the data of a set **role** of *roles*, together with an LTS $\mathcal{U}$ labeled with pairs $(\rho, \alpha^?) \in \mathbf{role} \times \mathbf{Usract}^?$.

The motivation for these two definitions is that we can combine them together to yield a model that can be "tightly" refined by a distributed system, in the sense that their states are related by a concrete relation amenable to proving refinements with respect to using a logic such as Trillium.

Definition 5.3. Given a user model $\mathcal{U}$ and a system model $\mathcal{N}$ over the same Trillium language instantiation, their *synchronized product* $\mathcal{U} \ltimes \mathcal{N}$ is an LTS labeled with $(\mathbf{role} \times (\mathbf{Usract}^?)) \cup \mathbf{Sysact}$, whose states are pairs (m, n) of states of $\mathcal{U}$ and of $\mathcal{N}$, respectively, together with the transitions:

$$\frac{m_1 \xrightarrow{(\rho,\bot)} m_2}{(m_1, n) \xrightarrow{(\rho,\bot)} (m_2, n)} \qquad \frac{m_1 \xrightarrow{(\rho,\alpha)} m_2 \quad n_1 \xrightarrow{\alpha} n_2}{(m_1, n_1) \xrightarrow{(\rho,\alpha)} (m_2, n_2)} \qquad \frac{n_1 \xrightarrow{\beta} n_2}{(m, n_1) \xrightarrow{\beta} (m, n_2)}$$

They correspond, respectively, to an actionless step of the user model, a joint action step of the user model and system model, and an autonomous step of the system.

In summary, internal program steps, *i.e.* when the action is not present, $\alpha^? = \bot$, do not synchronize with the system, and neither do the autonomous system steps with the program. The two sides synchronize via program actions, which correspond to both a program step and a system step.

5.2.1 The Abstract Network Model. In Fairneris, we instantiate the system model with the *network model* $\mathcal{N}et$. This network model is more abstract than the one used in the operational semantics of AnerisLang, as it eschews socket handlers. The states $(\mathsf{sm}, \mathcal{M})$ of $\mathcal{N}et$ are simply maps sm from socket addresses to lists of messages, representing the receive buffers, as well as the global message soup $\mathcal{M}$. The labels are the system actions of AnerisLang, described in §5.1. The transitions of this LTS are defined in a very similar way as the low-level network semantics. Thus, we omit them in the paper. The appSys function is defined in the obvious way, for example the case of message deliveries is the following:

$$\mathsf{appSys}((\mathsf{sm}, \mathcal{M}), \mathsf{Deliver}(msg)) \;=\; (\mathsf{sm}[sa := msg{::}\mathsf{sm}[sa]], \mathcal{M} \setminus \{msg\})$$

where sa is the destination socket address of msg, the message being delivered, and :: is the *cons* operation on lists.

To build the refinement between the program and the model, we maintain a relationship env_match between the two representations of the network: an AnerisLang network $(\mathcal{S}, \mathcal{M})$ refines a model network $(\mathsf{sm}, \mathcal{M}')$ when

$$\mathcal{M} = \mathcal{M}' \tag{1}$$

$$\mathcal{S}[ip] = S \;\wedge\; S[sh] = (skt, buf) \implies \mathsf{sm}[skt.sa] = buf \tag{2}$$

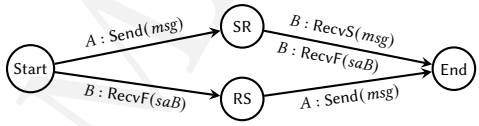
$$\mathsf{sm}[sa] = buf \implies \exists sh, skt.\;\; \mathcal{S}[ip][sh] = (skt, buf) \;\wedge\; skt.sa = sa \tag{3}$$

Intuitively, the message soups have to match exactly, and the receive buffers have to contain the same messages. Note that there could be an empty receive buffer in the AnerisLang network state that is not allocated at all in the model network state, this allows $\rightarrow_{\mathsf{prg}}$ transitions such as creating a new socket to *not* be reflected in the abstract model.

5.3 Obtaining the User Model Trace

Recall the user model of the retransmit example in Figure 1 from §2. The transitions in user models are annotated with two pieces of information: a role and an action. Roles and actions facilitate hiding scheduling details and network details, respectively To better understand, and appreciate the subtlety of obtaining a trace in $\mathcal{U}$ from a trace of $\mathsf{Fuel}(\mathcal{U} \ltimes \mathcal{N}et)$, let us first for now ignore the Fuel construction and focus on the difference between traces of $\mathcal{U} \ltimes \mathcal{N}et$ and $\mathcal{U}$.

The subtle relation between traces of $\mathcal{U} \ltimes \mathcal{N}et$ and $\mathcal{U}$. Let use consider a very simple program consisting of two nodes each with a single thread where one performs a single **send** operation and the other performs a single **recv** operation. This distributed program obviously terminates after performing exactly one send and one receive operation (which may fail or succeed). Let us consider the following simple user model for this distributed program:



$$msg \triangleq (saA, \texttt{"x"}, saB)$$

As expected, the user model above has finitely many traces, with a maximum length of two. However, the model $\mathcal{N}et$, and therefore also $\mathcal{U} \ltimes \mathcal{N}et$, has infinitely many states and infinitely many traces of arbitrary length including infinite traces. This is because $\mathcal{N}et$ must reflect all the non-deterministic behavior of the network. Below, are a few of these traces (we omit states):

$$\cdot \xrightarrow{\mathsf{Send}(msg)} \cdot \xrightarrow{\mathsf{Deliver}(msg)} \cdot \xrightarrow{\mathsf{RecvS}(msg)} \cdot \tag{4}$$

$$\cdot \xrightarrow{\mathsf{Send}(msg)} \cdot \xrightarrow{\mathsf{Dup}(msg)} \cdot \xrightarrow{\mathsf{Deliver}(msg)} \cdot \xrightarrow{\mathsf{Deliver}(msg)} \cdot \xrightarrow{\mathsf{RecvS}(msg)} \cdot \tag{5}$$

$$\cdot \xrightarrow{\mathsf{Send}(msg)} \cdot \xrightarrow{\mathsf{Dup}(msg)} \cdot \xrightarrow{\mathsf{Deliver}(msg)} \cdot \xrightarrow{\mathsf{Drop}(msg)} \cdot \xrightarrow{\mathsf{RecvS}(msg)} \cdot \tag{6}$$

MANUSCRIPT

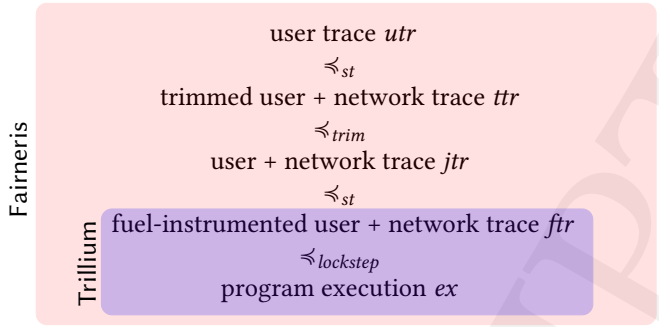


Fig. 4. The trace refinements involved in Fairneris

$$\cdot \xrightarrow{\mathsf{Send}(msg)} \cdot \xrightarrow{\mathsf{Dup}(msg)} \cdot \xrightarrow{\mathsf{Deliver}(msg)} \cdot \xrightarrow{\mathsf{Dup}(msg)} \cdot \xrightarrow{\mathsf{RecvS}(msg)} \cdot \xrightarrow{\mathsf{Drop}(msg)} \cdot \tag{7}$$

$$\cdot \xrightarrow{\mathsf{Send}(msg)} \cdot \xrightarrow{\mathsf{Dup}(msg)} \cdot \xrightarrow{\mathsf{Deliver}(msg)} \cdot \xrightarrow{\mathsf{RecvS}(msg)} \cdot \xrightarrow{\mathsf{Dup}(msg)} \cdot \xrightarrow{\mathsf{Deliver}(msg)} \cdot \xrightarrow{\mathsf{Dup}(msg)} \cdot \xrightarrow{\mathsf{Drop}(msg)} \cdots \tag{8}$$

In fact all of these traces correspond to the following $\mathcal{U}$ trace:

$$\mathsf{Start} \xrightarrow{\mathsf{Send}(msg)} \mathsf{SR} \xrightarrow{\mathsf{RecvS}(msg)} \mathsf{End} \tag{9}$$

Observe the difference between the $\mathcal{U}$ trace (9) and traces (4–8). The difference is in actions performed by the network (no program involvement): dropping, delivering, and duplicating *msg*. We call these system actions. There are two kinds of system actions: relevant ones, those after which there is a program step, and irrelevant ones, those that appear at the end of the trace. In (4), (5), and (6) all system actions are relevant while in (7) and (8) the system actions that happen after RecvS(*msg*) are irrelevant; infinitely many in case of (8).

Refinement tower. The refinement between a program execution *ex* and a trace *utr* of the user model $\mathcal{U}$ is defined in several steps, which are depicted in Figure 4. Going bottom up, the Trillium adequacy theorem ensures there is a lockstep forward simulation between the program execution *ex* and a trace *ftr* of $\mathsf{Fuel}(\mathcal{U} \ltimes \mathcal{N}et)$, where the Fuel-construction, described in Fairis [32], provides finite stuttering and scheduler fairness preservation, in the sense that if *ex* is fair (w.r.t. thread ids), then *ftr* is fair w.r.t. *roles*. This step will be discussed in §5.4.

Removing finite stutter from *ftr* yields a trace *jtr* of the "joint model" $\mathcal{U} \ltimes \mathcal{N}et$. Extracting a trace *utr* of the user model $\mathcal{U}$ from *jtr* is done in two steps: first the trailing network step (irrelevant system actions) are removed in a process we call *trimming*, resulting in a trace *ttr* of $\mathcal{U} \ltimes \mathcal{N}et$. Finally, the resulting trimmed trace *ttr* is projected onto $\mathcal{U}$, yielding the user model trace *utr*, that the user can use to reason about the distributed system — during this step we also remove all the relevant system actions.

The important properties of all these refinements is that:

(i) They preserve (scheduler and network) fairness in the upward direction (according to Figure 4) so that if the execution *ex* is fair, then so is *utr*, and
(ii) They all downwards-preserve liveness properties of interest, such as LTL formulas of the form $\mathbb{F}P$, where P only depends on the current network action.

For example, this allows us to conclude that the program of Figure 1 eventually sends the "done!" message from the fact that the model does.

Here, we describe the two types of refinement that are involved in the outer region of Figure 4, and how they are obtained in a generic way without any additional proof burden on the user.

Trimming refinement. The traces we consider contain both transitions coming from the user model $\mathcal{U}$ and transitions coming from the network model $\mathcal{N}et$. The first step to extract the underlying user trace is to remove all trailing network steps, a process we call *trimming*. In a trimmed trace, there are finitely many network transitions before the next user transition. This allows us to define a productive function that removes network transitions, as described below.

Among the three kinds of refinement we use, trimming is the worst-behaved with respect to preservation of properties, as it is very asymmetric. For example, if $tr_1 \preccurlyeq_{trim} tr_2$, and if $tr_2 \Vdash \mathbb{F}P$, then $tr_1 \Vdash \mathbb{F}P$, but the converse is false. However, this property is sufficient for property (ii) above. Fairness preservation is proved with an ad-hoc reasoning which depends on the nature of the atomic predicates involved.

Stutter refinement. According to the standard definition, the trace t_1 *stutter-refines* t_2 if t_2 can be obtained from t_1 by removing finite stutter, where a finite stutter is a finite sequence of repeated states. For instance, the sequence $122333\cdots$ stutter-refines $123\cdots$. In our setting however, where the two traces are equipped with states and labels of different nature, the definition sketched above is not applicable. Our solution is to consider a *heterogeneous* notion of stutter-equivalence, between traces with different states S_1, S_2 and labels L_1, L_2. This notion of refinement is parameterized by a map $\pi_S : S_1 \to S_2$ between states and a partial map $\pi_L : L_1 \rightharpoonup L_2$ between labels. The idea is that a step $s_1 \xrightarrow{\ell} s_1'$ in a trace t_1 is a stutter step iff $\pi_S(s_1) = \pi_S(s_2)$ and if $\pi_L(\ell)$ is not defined. Formally, it is defined as follows, where only the rightmost rule can be used infinitely many times in a row.[2]

$$\frac{\pi_S(s_1) = s_2}{\langle s_1 \rangle \preccurlyeq_{st} \langle s_2 \rangle} \qquad \frac{\pi_S(s_1) = \pi_S(\mathsf{hd}(t_1)) = \mathsf{hd}(t_2) \quad \pi_L(\ell_1) = \bot \quad t_1 \preccurlyeq_{st} t_2}{s_1 \xrightarrow{\ell_1} t_1 \preccurlyeq_{st} t_2} \qquad \frac{\pi_S(s_1) = s_2 \quad \pi_L(\ell_1) = \ell_2 \quad t_1 \preccurlyeq_{st} t_2}{s_1 \xrightarrow{\ell_1} t_1 \preccurlyeq_{st} s_2 \xrightarrow{\ell_2} t_2}$$

In practice, we only have the trace t_1 and we wish to obtain a trace t_2 such that $t_1 \preccurlyeq_{st} t_2$. This is possible when the set of traces with states in S_1 and labels in L_1 is equipped with a "potential" function $\Psi : \mathbf{Trace}(S_1, L_1) \to \mathbb{N}$ that decreases along stutter transitions. We use this technique twice: once for removing the fuel-instrumentation, as in Fairis, and once for obtaining the user trace from the trimmed trace *ttr*. In the latter case, network transitions are the stutter transitions, as they cannot modify the user state — see the construction of $\mathcal{U} \ltimes \mathcal{N}et$ explained below. To define the potential Ψ, we rely on the fact that the traces we consider are trimmed, which means that there can only be finitely many network transitions before the next user transition. Thus, we define $\Psi(ttr)$ to be the number of network transitions before the first program step.

To reason about property preservation along stutter refinement, we defined the predicate

$$P \lesssim_{st} Q \iff \forall tr_1, tr_2.\ \ tr_1 \preccurlyeq_{st} tr_2 \Rightarrow (tr_1 \Vdash P \Leftrightarrow tr_2 \Vdash Q)$$

which, intuitively, states that P corresponds to Q when going through a stutter refinement in either direction. It is a congruence with respect to LTL modalities we use; for example, $P \lesssim_{st} Q$ implies $\mathbb{F}P \lesssim_{st} \mathbb{F}Q$, and $\mathbb{G}P \lesssim_{st} \mathbb{G}Q$. However, for point predicates P and Q, for example those which only depend on the current label of the trace, it is not obvious how to relate them under $\lesssim_{st}$. Nevertheless, the $\mathbb{F}$ modality alleviates this issue because we can show the following:

$$(\forall \ell,\ P(\ell) \Leftrightarrow (\pi_L(\ell) \neq \bot \wedge Q(\pi_L(\ell)))) \quad \Longrightarrow \quad \mathbb{F}P \ \lesssim_{st} \ \mathbb{F}Q$$

As it turns out, this simpler result is sufficient to prove the properties (i) and (ii) for $\preccurlyeq_{st}$.

Network fairness preservation. As the user models are oblivious to the autonomous events of the network such as deliveries, its formulation of network fairness cannot have the same shape as that of the concrete network, as it relates sending messages to deliveries. Instead user models can rely on (ModNetFair). We prove that our notion of refinement preserves network fairness; that if *ex* refines *utr*, and if *ex* satisfies (NetFair), then *utr* satisfies (ModNetFair). To do so, we first prove

[2]This definition should be understood as an inductive relation nested in a co-inductive relation.

that (NetFair) holds on the joint-model trace *jtr*, and that it implies that (ModNetFair) also holds for *jtr*, by first using (NetFair) to prove that a message is eventually delivered (and this is at the tail of the corresponding receive buffer), and then the assumption of (ModNetFair) that the program receives on that socket address infinitely often to conclude that the message is eventually received.

5.4 Model of the Logic

We conclude this section with how the Fairneris logic is defined. First we explain the concept of *trace interpretation* [32] and define that of Fairneris. This allows us to explicate the definitions of the predicates $\mathsf{wp}\ e\ \langle \Phi \rangle$ and $\mathsf{mu}^{\zeta;\alpha^?}$, presented in §3, which manipulate this trace interpretation. Finally, we sketch how the first refinement in Figure 4 is obtained using the Trillium logic.

Trace interpretation. The general weakest precondition of Trillium is parameterized by a so-called *trace interpretation* predicate, an Iris predicate relating traces of the program to the traces of the model (This is a generalization of what Iris calls a state interpretation). We write $TI(\tau, \kappa)$. In our instantiation of Trillium with AnerisLang, the arguments τ and κ are respectively, a finite execution of the program (a sequence of *Cfg*'s to be more precise), and a finite trace of the global model $\mathsf{Fuel}(\mathcal{U} \ltimes \mathcal{N}et)$. These are the histories of the program and of the model, and are intuitively prefixes of the full executions *ex* and user model traces *utr* we considered above. Intuitively, $TI(\tau, \kappa)$ states that the current state of the program is $last(\tau)$, the current state of the model is $last(\kappa)$, and that they match, in the sense determined by the person instantiating Trillium who picks the trace interpretation relation. Note that, since Trillium is designed to build lockstep simulations, the two histories τ and κ must always have the same length. For the Fairneris instantiation of Trillium, the trace interpretation is defined as follows:

$$TI(\tau, \kappa) \quad := \quad \mathsf{traces_match}(\tau, \kappa) \ * \ \mathsf{aneris_interp}(\tau) \ * \ \mathsf{model_interp}(last(\tau), last(\kappa))$$

where aneris_interp is the existing Aneris state/trace interpretation, reflecting the state of node heaps, sockets, and the message soup in Iris resources. The traces_match predicate is a straightforward modification of that same predicate from Fairis, adding support for actions in the operational semantics of the programming language and in the model. Here, traces_match also adds the requirement that actions must match exactly in the two traces, allowing us to transport knowledge about the network actions back-and-forth between the distributed system and the abstract model ($\mathcal{U} \ltimes \mathcal{N}et$). Finally, the relation $\mathsf{model_interp}((tp, \sigma), \tilde{m})$ asserts that the current state of the model is $\tilde{m}$, and that it is well-formed. The state $\tilde{m} \in \mathsf{Fuel}(\mathcal{U} \ltimes \mathcal{N}et)$ can be decomposed into three parts, that are each treated differently:

- The fuel-instrumentation is handled as in Fairis [32]
- The state m of the user model $\mathcal{U}$ is owned by the predicate TI[3]
- A novel network part $\mathsf{n} \in \mathcal{N}et$: the trace interpretation TI asserts $\mathsf{env_match}(\sigma, \mathsf{n})$ holds.

The single-step weakest precondition (SSWP). One of the main innovation of Fairneris over Fairis in terms of their program logics is the logical rules governing the *model update* $\mathsf{mu}^{\zeta;\alpha^?}$, and how it handles lockstep refinement by working in tandem with the *single step weakest precondition* $\mathsf{wp}\ e\ \langle \Phi \rangle$ (abbreviated SSWP), a slight variation of a concept introduced by Fairis. In Fairis[32], SSWP was intuitively defined as follows: during a single step of the program, given the current trace interpretation $TI(\tau, \kappa)$, we can update the ghost state in such a way that the trace interpretation for the extended histories $TI(\tau \xrightarrow{\zeta} (tp', \sigma'), \kappa \xrightarrow{\ell} m')$holds. As such, updating the ghost state for the program and for the model $\mathcal{M}$ had to be done at the same time.

[3]For readers familiar with Iris, TI asserts ownership of the authoritative part of the resource algebra $\textsc{Auth}(\textsc{Ex}(\mathcal{U}))$

Trillium SSWP

$$\mathsf{wp}\ e\ \langle \Phi \rangle \triangleq \Rrightarrow \big(e \notin \mathit{Val} * \forall \tau, \kappa, \sigma, K, \mathsf{tp}.\ \mathit{last}(\tau) = (\mathsf{tp}[\zeta \mapsto K[e]], \sigma) * \mathit{TI}(\tau, \kappa) \mathrel{-\!\!*} \Rrightarrow$$
$$\mathit{reducible}(e, \sigma) * \triangleright \forall \alpha^?, e', \sigma', \vec{e}.\ (e, \sigma) \xrightarrow{\alpha^?}_{\mathsf{prg}} (e', \sigma'; \vec{e}) \mathrel{-\!\!*} \Rrightarrow$$
$$\mathit{TI}'(\tau \xrightarrow{(\zeta, \alpha^?)} (\mathsf{tp}[\zeta \mapsto K[e']], \sigma'), \kappa, \zeta, \alpha^?) * \Phi\ (e', \alpha^?) * \vec{e} = \epsilon\big)$$

Trillium Model Update

$$\mathsf{mu}^{\zeta;\alpha^?}\ P \triangleq \forall \tau, \kappa.\ \mathit{TI}'(\tau, \kappa, \zeta, \alpha^?) \mathrel{-\!\!*} \Rrightarrow \exists \mathsf{m} \in \mathcal{M}, \rho.\ \ \mathit{last}(\kappa) \xrightarrow{\rho} \mathsf{m} * \mathit{TI}(\tau, \kappa \xrightarrow{\rho} \mathsf{m}) * \Rrightarrow P$$

System weakest precondition

$$\mathsf{system_wp} \triangleq \forall \tau, \kappa, \beta, \mathit{tp}, \sigma. \Big(\big(\mathit{last}(\tau) \xrightarrow{\beta}_{\mathit{sys}} (\mathit{tp}, \sigma)\big)\ *\ \mathit{TI}(\tau, \kappa)\Big) \mathrel{-\!\!*} \Rrightarrow \exists \mathsf{m}, \ell.\ \mathit{TI}\big(\tau \xrightarrow{\beta} (\mathit{tp}, \sigma), \kappa \xrightarrow{\ell} \mathsf{m}\big)$$

Fig. 5. Definitions of basic constructs

An updated definition of the SSWP, and the novel definition of the model update, is given in Figure 5. Here, we make use of a so-called *torn trace interpretation* $\mathit{TI}'(\tau, \kappa, \zeta, \alpha^?)$, which expresses that the program history τ is one step ahead of the model history κ, for a step that is labeled by $(\zeta, \alpha^?)$. Formally, it is defined as:

$$\mathit{TI}'(\tau, \kappa, \zeta, \alpha^?) \ \triangleq\ \exists \tau', \mathit{tp}, \sigma.\ \ \tau = \tau' \xrightarrow{(\zeta, \alpha^?)} (\mathit{tp}, \sigma) *$$
$$\mathsf{traces_match}(\tau', \kappa) * \mathsf{aneris_interp}(\tau') * \mathsf{model_interp}(\mathit{last}(\tau'), \mathit{last}(\kappa))$$

The Aneris part of the ghost state corresponds to extended trace τ of the program, whereas the current model history corresponds to the immediate prefix τ' of the program history. Thus, the SSWP moves the program execution by one step, tearing the relationship between the traces of the program and the model. The model update mends this tear, by moving the model trace one step so that the traces match again. This explains the inference rule WP-STEP, which decomposes a weakest precondition into (1) an SSWP handling the next step of the program, (2) a model update handling the next step of the model, and (3) a weakest precondition handling the remainder of the steps of both the program and the model. An important thing to note is that we can open invariants around the composite step of the SSWP and MU, necessary for accessing and updating an invariant-governed model state over the entire lockstep. This is made possible by the first and last $\Rrightarrow$ of the SSWP and MU, respectively. As previously stated, we omit details about masks, but refer the interested reader to our accompanying artifact [3].

The system weakest precondition. The intuitive reason $\mathsf{wp}^{\zeta}\ e\ \{\Phi\}$ guarantees safety of the program e is that it states that every step of the *program* is safe, in the sense that the program should not crash (should not get stuck), and that it preserves the trace interpretation. In the case of distributed systems that is not enough, as the *network* also takes *autonomous* steps. Similarly, these steps must also preserve the trace interpretation.

The solution in Trillium [32] was to introduce the *system weakest precondition* (system WP), which simply states that all steps $\sigma \rightarrow_{\mathit{sys}} \sigma'$ of the network preserve the trace interpretation, as shown in Figure 5. Unlike weakest preconditions that are proved by the users of a program logic like Fairneris based on the Trillium framework, the system WP is proved once and for all by the person instantiating Trillium, and does not appear at all to the user.

In the case of Fairneris this step is more difficult than for the previous Aneris instantiation of Trillium by Timany et al. [32], as we additionally need to prove that each network step refines the

	Fairneris (this work)	Aneris [15]	Disel [27]	Grove [28]	Igloo [30]	IronFleet [11]	Verdi [33]
Higher-Order	✓	✓	✓	✓	✓	✓	✓
Concurrency	✓	✓	✓	✓	✓	✓	✓
Safety properties	✓	✓	✓	✓	✓	✓	✓
Liveness properties	✓	✓	✓	✓	✓	✓*	✓
Refinement-based	✓	✓†	✓	✓	✓	✓	✓‡
Foundational	✓	✓	✓	✓	✓§	✓	✓¶
Crash Recovery & Reconfiguration	✓	✓	✓	✓	✓	✓	✓

*Requires a special program shape: a main event loop.
†The version by Timany et al. [32] is refinement-based; albeit only for safety properties.
‡Refinement between programs; initially programs are written against an idealized network model and then automatically refined to programs that assume a less reliable network model.
§Meta theory proven correct in the Isabelle proof assistant [1]; programs proven correct in Verifast [13] and Negini [9].
¶Relies on a not-so-small shim to run programs written in Verdi's DSL.

Table 1. Comparison of different approaches to reasoning about distributed systems' implementations.

corresponding step in the abstract network model $\mathcal{N}et$. This is one of the places the gap between the concrete network and abstract model of the network needs to be bridged, by proving that properties (1–3) are preserved.

The lockstep refinement using Trillium. Fairneris is based on Trillium [32], and thus inherits its weakest precondition $\mathsf{wp}^{\zeta}\ e\ \{\Phi\}$, almost directly. We remark that we had to update the Trillium weakest precondition to work on operational semantics and traces with action labels, and reprove, albeit trivial, its adequacy theorem. The first step of the proof of Fairneris adequacy (Theorem 3.1) is to apply the updated Trillium adequacy theorem, Theorem 5.4. Below, we will present and explain this theorem and sketch how we used it to prove the Fairneris adequacy theorem.

Theorem 5.4 (Trillium adequacy). *Let e be a program, σ a program state, ζ the locale of e, and Φ an Iris predicate on values. Let $\mathsf{m} \in \mathcal{M}$ be a model state and ξ a relation on finite traces of the program and the model. Let $(\{\zeta \mapsto e\}, \sigma)$ be the initial configuration of the program. If $\xi((\{\zeta \mapsto e\}, \sigma), \mathsf{m})$ holds for the initial singleton traces, and furthermore we have*

$$\Vdash\ TI(c, \mathsf{m}) * wp^{\zeta}\ e\ \{\Phi\} * AlwaysHolds_{\mathrm{T}}(\xi, c, \mathsf{m})$$

then $(\{\zeta \mapsto e\}, \sigma) \precsim_{\xi} \mathsf{m}$ holds in the meta-logic.

The predicate $(\{\zeta \mapsto e\}, \sigma) \precsim_{\xi} \mathsf{m}$ states that there exists a lockstep forward simulation that preserves the relation ξ, and which relates $(\{\zeta \mapsto e\}, \sigma)$ and m. The $AlwaysHolds_{\mathrm{T}}$ predicate is analogous to the corresponding predicate in Theorem 3.1. The refinement $\preccurlyeq_{lockstep}$ at the bottom of Figure 4 is obtained from $\precsim_{\xi}$ using a general result stating that such simulation relations induce a refinement relation between possibly-infinite traces [32]. All that remains is to apply Theorem 5.4 with of $\mathcal{M} \coloneqq \mathsf{Fuel}(\mathcal{U} \ltimes \mathcal{N}et)$, where $\mathcal{U}$ is the model chosen by the user of Fairneris and with a well-chosen relation ξ, which incorporates the user-picked model invariant Ψ. The Fuel construction, which is taken essentially verbatim from Fairis [32], together with ξ ensure that $\precsim_{\xi}$ preserves fairness. We remark that this is more difficult than for the other refinement relations, because fairness of the program execution is defined in terms of *locales*, whereas the fairness for traces of $\mathsf{Fuel}(\mathcal{U} \ltimes \mathcal{N}et)$ is defined in terms of the *roles* of $\mathcal{U}$.

6 Related and Future Work

Liveness Reasoning for Higher-Order Concurrent Programs. In addition to Trillium, that our work is based on and we have discussed at length already, the only other works on verifying

liveness of higher-order concurrent programs are NoLa [23] and LiLo [19]. Neither of these approaches have been used to reason about distributed systems. Both of these works are based on the idea of abandoning step-indexing and hence impredicative invariants. Instead, these systems shallowly embed the syntax of the logic (in Rocq) to construct a *predicative* higher-order logic, which is known to have modularity limitations when reasoning about higher-order programs [31]. For one, in prior work on Aneris, the socket protocols that we inherit use impredicativity to send other socket protocols over the network. NoLa only support reasoning about termination while LiLo supports other liveness properties by proving liveness-preserving refinements between the program and another simpler program. Because it relies on FOS [18], Lilo is restricted in the fairness assumptions it can express; in particular, we do not believe it is capable of expressing ModNetFair.

Verification of Distributed Systems Implementations. A summary of the comparison of our work with other distributed systems verification approaches, *i.e.*, Aneris [15], Disel [27], Grove [28], Igloo [30], IronFleet [11], and Verdi [33], is given in Table 1.

The only program logics other than Aneris that support reasoning about higher-order and concurrent distributed systems are Grove [28] and Igloo [30]. The approach of Grove [28] to verifying distributed systems is very close to that of Aneris [15] upon which we have based our work — Grove is also based on the Iris framework [14]. It pushes the limits of this methodology by verifying (safety properties of) crash recovery (some nodes may spontaneously fail and restart) and reconfiguration (nodes can dynamically join or leave the distributed system). An interesting and important future work is to prove liveness properties of such distributed systems. Indeed, one of the main reasons for implementing crash recovery and dynamic reconfiguration in distributed systems is to improve availability. We believe that the Fairneris methodology developed here should also scale to supporting reasoning about these more advanced features of distributed systems. Regarding Igloo, we remark that it proves correctness of distributed systems by establishing a rather weak refinement relation between the program and a transition system which only establishes that the program and the transition system perform the same I/O operations. The weakness of Igloo's refinement relation enables great flexibility with respect to program structure, thus enabling it to reason about concurrent programs that use higher-order programming features which, unlike, *e.g.*, IronFleet, are not written as simple event loops. However, it is not clear how the methodology of Igloo [30] can be extended to support reasoning about liveness properties.

Verification of Liveness Properties of Distributed Protocols. Several works [4, 5, 8, 22, 24–26, 34] have considered proving liveness properties of distributed systems at the protocol level by studying these protocols as state transition systems at various levels of detail. Most of these approaches focus on specifically proving correctness of consensus protocols. They often [8] even require the program to be constructed in terms of rounds of reaching consensus. In many cases [4, 5, 22, 24, 25] these works verify liveness properties by reducing liveness properties to safety properties which they then automate. Compared to our approach of verifying liveness properties of concrete implementations, these approaches scale better, *e.g.*, to proving liveness in the presence of byzantine faults [4, 5, 22, 26]. Proving liveness of distributed systems in the presence of byzantine faults is another important and interesting future work. We believe that the state-transition-systems style protocols studied by these works are interesting candidates to consider as high-level models in Fairneris. This would allow us to scale Fairneris's approach to reasoning about liveness in the presence of byzantine faults.

7 Conclusion

We have presented Fairneris, the first higher-order concurrent separation logic that is capable of reasoning about both the safety and liveness of concrete implementations of distributed systems, by

following the methodology of Fairis and Trillium, *i.e.*, via constructing and exploiting a refinement relation between the distributed program and an abstract model of the distributed algorithm it implements. Using this method, the program logic handles what it does best — the complex semantics of the language and the safety properties — while the liveness reasoning happens on a high-level representation of the system.

References

[1] 2002. *Isabelle/HOL*. Springer Berlin Heidelberg. doi:10.1007/3-540-45949-9

[2] E. A. Akkoyunlu, K. Ekanadham, and R. V. Huber. 1975. Some constraints and tradeoffs in the design of network communications. In *Proceedings of the Fifth ACM Symposium on Operating Systems Principles* (Austin, Texas, USA) *(SOSP '75)*. Association for Computing Machinery, New York, NY, USA, 67–74. doi:10.1145/800213.806523

[3] Anonymous Authors. 2026. Rocq Mechanization of the paper.

[4] Idan Berkovits, Marijana Lazić, Giuliano Losa, Oded Padon, and Sharon Shoham. 2019. Verification of Threshold-Based Distributed Algorithms by Decomposition to Decidable Logics. In *Computer Aided Verification*, Isil Dillig and Serdar Tasiran (Eds.). Springer International Publishing, Cham, 245–266.

[5] Nathalie Bertrand, Vincent Gramoli, Igor Konnov, Marijana Lazić, Pierre Tholoniat, and Josef Widder. 2022. Holistic Verification of Blockchain Consensus. In *36th International Symposium on Distributed Computing (DISC 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 246)*, Christian Scheideler (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 10:1–10:24. doi:10.4230/LIPIcs.DISC.2022.10

[6] Tushar D. Chandra, Robert Griesemer, and Joshua Redstone. 2007. Paxos made live: an engineering perspective. In *Proceedings of the Twenty-Sixth Annual ACM Symposium on Principles of Distributed Computing* (Portland, Oregon, USA) *(PODC '07)*. Association for Computing Machinery, New York, NY, USA, 398–407. doi:10.1145/1281100.1281103

[7] Emanuele D'Osualdo, Julian Sutherland, Azadeh Farzan, and Philippa Gardner. 2021. TaDA Live: Compositional Reasoning for Termination of Fine-grained Concurrent Programs. *ACM Trans. Program. Lang. Syst.* (2021). doi:10.1145/3477082

[8] Cezara Drăgoi, Thomas A. Henzinger, and Damien Zufferey. 2016. PSync: a partially synchronous language for fault-tolerant distributed algorithms. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (St. Petersburg, FL, USA) *(POPL '16)*. Association for Computing Machinery, New York, NY, USA, 400–415. doi:10.1145/2837614.2837650

[9] Marco Eilers and Peter Müller. 2018. Nagini: A Static Verifier for Python. In *Computer Aided Verification*, Hana Chockler and Georg Weissenbacher (Eds.). Springer International Publishing, Cham, 596–603.

[10] Léon Gondelman, Jonas Kastberg Hinrichsen, Mário Pereira, Amin Timany, and Lars Birkedal. 2023. Verifying Reliable Network Components in a Distributed Separation Logic with Dependent Separation Protocols. *Proc. ACM Program. Lang.* 7, ICFP (2023), 847–877. doi:10.1145/3607859

[11] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. 2015. IronFleet: proving practical distributed systems correct. In *Proceedings of the 25th Symposium on Operating Systems Principles* (Monterey, California) *(SOSP '15)*. Association for Computing Machinery, New York, NY, USA, 1–17. doi:10.1145/2815400.2815428

[12] Gerard J. Holzmann. 1997. The Model Checker SPIN. *IEEE Trans. Software Eng.* 23, 5 (1997), 279–295. doi:10.1109/32.588521

[13] Bart Jacobs, Jan Smans, Pieter Philippaerts, Frédéric Vogels, Willem Penninckx, and Frank Piessens. 2011. VeriFast: A Powerful, Sound, Predictable, Fast Verifier for C and Java. In *NASA Formal Methods*, Mihaela Bobaru, Klaus Havelund, Gerard J. Holzmann, and Rajeev Joshi (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 41–55.

[14] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20. doi:10.1017/S0956796818000151

[15] Morten Krogh-Jespersen, Amin Timany, Marit Edna Ohlenbusch, Simon Oddershede Gregersen, and Lars Birkedal. 2020. Aneris: A Mechanised Logic for Modular Reasoning about Distributed Systems. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings*. 336–365. doi:10.1007/978-3-030-44914-8_13

[16] L. Lamport. 1977. Proving the Correctness of Multiprocess Programs. *IEEE Transactions on Software Engineering* SE-3, 2 (1977), 125–143. doi:10.1109/TSE.1977.229904

[17] Leslie Lamport. 1992. Hybrid Systems in TLA$^+$. In *Hybrid Systems*, Robert L. Grossman, Anil Nerode, Anders P. Ravn, and Hans Rischel (Eds.). Lecture Notes in Computer Science, Vol. 736. Springer, 77–102. doi:10.1007/3-540-57318-6_25

UNDER REVIEW

[18] Dongjae Lee, Minki Cho, Jinwoo Kim, Soonwon Moon, Youngju Song, and Chung-Kil Hur. 2023. Fair Operational Semantics. *Proc. ACM Program. Lang.* 7, PLDI, Article 139 (June 2023), 24 pages. doi:10.1145/3591253

[19] Dongjae Lee, Janggun Lee, Taeyoung Yoon, Minki Cho, Jeehoon Kang, and Chung-Kil Hur. 2025. Lilo: A Higher-Order, Relational Concurrent Separation Logic for Liveness. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 125 (April 2025), 28 pages. doi:10.1145/3720525

[20] Hongjin Liang and Xinyu Feng. 2016. A program logic for concurrent objects under fair scheduling. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, Rastislav Bodík and Rupak Majumdar (Eds.). ACM, 385–399. doi:10.1145/2837614.2837635

[21] Zongyuan Liu, Sergei Stepanenko, Jean Pichon-Pharabod, Amin Timany, Aslan Askarov, and Lars Birkedal. 2023. VMSL: A Separation Logic for Mechanised Robust Safety of Virtual Machines Communicating above FF-A. *Proc. ACM Program. Lang.* 7, PLDI, Article 165 (June 2023), 25 pages. doi:10.1145/3591279

[22] Giuliano Losa and Mike Dodds. 2020. On the Formal Verification of the Stellar Consensus Protocol. In *2nd Workshop on Formal Methods for Blockchains (FMBC 2020) (Open Access Series in Informatics (OASIcs), Vol. 84)*, Bruno Bernardo and Diego Marmsoler (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:9. doi:10.4230/OASIcs.FMBC.2020.9

[23] Yusuke Matsushita and Takeshi Tsukada. 2025. Nola: Later-Free Ghost State for Verifying Termination in Iris. *Proc. ACM Program. Lang.* 9, PLDI, Article 151 (June 2025), 27 pages. doi:10.1145/3729250

[24] Kenneth L. McMillan and Oded Padon. 2020. Ivy: A Multi-modal Verification Tool for Distributed Algorithms. In *Computer Aided Verification: 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part II* (Los Angeles, CA, USA). Springer-Verlag, Berlin, Heidelberg, 190–202. doi:10.1007/978-3-030-53291-8_12

[25] Oded Padon, Jochen Hoenicke, Giuliano Losa, Andreas Podelski, Mooly Sagiv, and Sharon Shoham. 2017. Reducing liveness to safety in first-order logic. *Proc. ACM Program. Lang.* 2, POPL, Article 26 (dec 2017), 33 pages. doi:10.1145/3158114

[26] Longfei Qiu, Yoonseung Kim, Ji-Yong Shin, Jieung Kim, Wolf Honoré, and Zhong Shao. 2024. LiDO: Linearizable Byzantine Distributed Objects with Refinement-Based Liveness Proofs. *Proc. ACM Program. Lang.* 8, PLDI, Article 193 (jun 2024), 25 pages. doi:10.1145/3656423

[27] Ilya Sergey, James R. Wilcox, and Zachary Tatlock. 2018. Programming and proving with distributed protocols. *Proc. ACM Program. Lang.* 2, POPL (2018), 28:1–28:30. doi:10.1145/3158116

[28] Upamanyu Sharma, Ralf Jung, Joseph Tassarotti, Frans Kaashoek, and Nickolai Zeldovich. 2023. Grove: a Separation-Logic Library for Verifying Distributed Systems. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 113–129. doi:10.1145/3600006.3613172

[29] Simon Spies, Lennard Gäher, Daniel Gratzer, Joseph Tassarotti, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. 2021. Transfinite Iris: resolving an existential dilemma of step-indexed separation logic. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 20211*. 80–95. doi:10.1145/3453483.3454031

[30] Christoph Sprenger, Tobias Klenze, Marco Eilers, Felix A. Wolf, Peter Müller, Martin Clochard, and David Basin. 2020. Igloo: Soundly Linking Compositional Refinement and Separation Logic for Distributed System Verification. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 152 (Nov. 2020), 31 pages. doi:10.1145/3428220

[31] Kasper Svendsen and Lars Birkedal. 2014. Impredicative Concurrent Abstract Predicates. In *Programming Languages and Systems*, Zhong Shao (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 149–168.

[32] Amin Timany, Simon Oddershede Gregersen, Léo Stefanesco, Jonas Kastberg Hinrichsen, Léon Gondelman, Abel Nieto, and Lars Birkedal. 2024. Trillium: Higher-Order Concurrent and Distributed Separation Logic for Intensional Refinement. *Proc. ACM Program. Lang.* POPL (2024). doi:10.1145/3632851

[33] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas E. Anderson. 2015. Verdi: a framework for implementing and formally verifying distributed systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015*, David Grove and Stephen M. Blackburn (Eds.). ACM, 357–368. doi:10.1145/2737924.2737958

[34] Jianan Yao, Runzhou Tao, Ronghui Gu, and Jason Nieh. 2024. Mostly Automated Verification of Liveness Properties for Distributed Protocols with Ranking Functions. *Proc. ACM Program. Lang.* 8, POPL, Article 35 (jan 2024), 32 pages. doi:10.1145/3632877

Manuscript